

Chapter 1 Introduction to Statistics

Welcome to your introduction to statistics. You will be learning the basics of statistics, along with applications of statistics within the R language. This book will provide fundamentals of the concepts and the code to apply these concepts in R. At the end of each section, you will also find the Python code that will perform a comparable analysis (however, in class, we will be discussing and focusing on the R code).

This Chapter aims to answer the following questions:

- What type of data is being analyzed?
 - Nominal
 - Ordinal
 - Continuous/Discrete
- How do we describe distributions of these variables?
 - Center
 - Spread
 - Shape
 - Graphical Display
- How do we create confidence intervals for parameters?
- How do we perform hypothesis testing?
 - One sample t-test
 - Two sample t-test
 - Testing Normality
 - Testing Equality of Variances
 - Testing Equality of Means
 - Mann-Whitney-Wilcoxon Test

The following packages will be used in this textbook (there are some sections that add more later in the textbook, so keep your eyes open for those). In order to use a library in the R session, you will first need to install it on your local drive (this only needs to be done once....however, you should periodically check for updates to packages). You should also periodically check to see if R or RStudio needs to be updated as well.

Installing packages:

```
install.packages('AmesHousing')
install.packages('tidyverse')
install.packages('car')
install.packages('DescTools')
install.packages('corrplot')
install.packages('mosaic')
install.packages('modelr')
install.packages('plotly')
install.packages('ggplot2')
install.packages('Hmisc')
install.packages('onehot')
install.packages('jmuOutlier')
install.packages('leaps')
install.packages('glmnet')
install.packages('nortest')
install.packages('lmtest')
install.packages('gmodels')
install.packages('TSA')
install.packages('carData')
```

You will need to bring the packages into your currently opened R session (this will need to be done whenever you open a new session):

```
library(AmesHousing)
library(tidyverse)
library(car)
library(DescTools)
library(corrplot)
library(mosaic)
library(modelr)
library(plotly)
library(ggplot2)
library(Hmisc)
library(onehot)
library(jmuOutlier)
library(leaps)
library(glmnet)
library(nortest)
library(lmtest)
library(gmodels)
library(TSA)
library(carData)
```

1.1 Exploratory Data Analysis (EDA)

The crucial first step to any data science problem is exploratory data analysis (EDA). Before you attempt to run any models, or jump towards any formal statistical analysis, you must **explore your data**. Many unexpected frustrations arise when exploratory analysis is overlooked; knowing your data is critical to your ability to make necessary assumptions about it and analyzing it appropriately. This preliminary analysis will help inform our decisions for data manipulation, give us a base-level understanding of our variables and the relationships

between them, and help determine which statistical analyses might be appropriate for the questions we are trying to answer. Some of the questions we aim to answer through exploratory analysis are:

- What kind of variables do you have?
 - Continuous
 - Nominal
 - Ordinal
- How are the attributes stored?
 - Strings
 - Integers
 - Floats/Numeric
 - Dates
- What do their distributions look like?
 - Center/Location
 - Spread
 - Shape
- Are there any anomalies?
 - Outliers
 - Leverage points
 - Missing values
 - Low-frequency categories

Throughout the textbook, we will continue to use a real-estate data set that contains the `sale_price` and numerous physical attributes of nearly 3,000 homes in Ames, Iowa in the early 2000s. You should have already loaded the `AmesHousing` package which has the data set, so now we will create a nicely formatted data set with the `make_ordinal_ames()` function. The function `str()` is a good way to see what variables are in the data set and how they are currently formatted. Take a look at some of the variables within the `AmesHousing` data set.

```
ames <- make_ordinal_ames()  
str(ames)
```

1.1.1 Types of Variables

The columns of a data set are referred to by the following **equivalent terms**:

- Variables
- Features
- Attributes
- Predictors/Targets
- Factors
- Inputs/Outputs

This book may use any of these words interchangeably to refer to a quality or quantity of interest in our data.

1.1.1.1 Nominal Variables

A **nominal** or **categorical variable** is a *quality of interest* whose values have no logical ordering. Color (“blue”, “red”, “green”...), ethnicity (“African-American”, “Asian”, “Caucasian”,...), and style of house (“ranch”, “two-story”, “duplex”, ...) are all examples of nominal attributes. The categories or values that these variables can take on - those words listed in quotes and parenthesis - are called the **levels** of the variable.

In modeling, nominal attributes are commonly transformed into **dummy variables**. Dummy variables are binary columns that indicate the presence or absence of a quality. There is more than one way to create dummy variables, and the treatment/estimates will be different depending on what type of model you are using. Linear regression models will use either **reference-level** or **effects coding**, whereas other machine learning models are more likely to use one-hot encoding or a variation thereof.

One-hot encoding

For machine learning applications, it is common to create a binary dummy column for each level of your categorical variable. This is the most intuitive representation of categorical information, answering indicative questions for each level of the variable: “*is it blue?*”, “*is it red?*” etc. The table below gives an example of some data, the original nominal variable (color) and the one-hot encoded color information.

Observation	Color	Blue	Red	Yellow	Other
1	Blue	1	0	0	0
2	Yellow	0	0	1	0
3	Blue	1	0	0	0
4	Red	0	1	0	0
5	Red	0	1	0	0
6	Blue	1	0	0	0
7	Yellow	0	0	1	0
8	Other	0	0	0	1

Table 1.1: One-hot dummy variable coding for the categorical attribute *color*

We will demonstrate the creation of this data using some simple random categorical data:

```
set.seed(41)
dat <- data.frame(y = c(rnorm(10,2), rnorm(10,1), rnorm(10,0)),
                  x1 = factor(rep(c("A", "B", "C"), each = 10)),
                  x2 = factor(rep(c("Z", "X", "Y", "W", "V", "U"), each = 5)))
print(dat)
```

```
##           y  x1 x2
```

## 1	1.2056317	A	Z
## 2	2.1972575	A	Z
## 3	3.0017043	A	Z
## 4	3.2888254	A	Z
## 5	2.9057534	A	Z
## 6	2.4936675	A	X
## 7	2.5992858	A	X
## 8	0.4203930	A	X
## 9	3.0006207	A	X
## 10	4.1880077	A	X
## 11	-0.2093244	B	Y
## 12	0.4126881	B	Y
## 13	2.0561206	B	Y
## 14	0.6834151	B	Y
## 15	0.9454590	B	Y
## 16	1.3297513	B	W
## 17	1.6630951	B	W
## 18	1.8783282	B	W
## 19	1.2028743	B	W
## 20	3.2744025	B	W
## 21	-0.8992970	C	V
## 22	2.1394903	C	V
## 23	-1.1659510	C	V
## 24	-0.0471304	C	V
## 25	0.4158763	C	V
## 26	1.7200805	C	U
## 27	-0.7843607	C	U
## 28	-1.3039296	C	U
## 29	-0.4520359	C	U
## 30	-1.7739919	C	U

Unlike reference and effects coding, which are used in linear regression models, one-hot encoding is most quickly achieved through use of the `onehot` package in R, which first creates an “encoder” to do the job quickly.

The speed of this function has been tested against both the base R `model.matrix()` function and the `dummyVars()` function in the `caret` package and is *substantially* faster than either.

```
library(onehot)
```

```
encoder = onehot(dat)
```

```
dummies = predict(encoder, dat)
```

```
head(dummies)
```

```
##           y x1=A x1=B x1=C x2=U x2=V x2=W x2=X x2=Y x2=Z
## [1,] 1.205632    1    0    0    0    0    0    0    0    1
## [2,] 2.197258    1    0    0    0    0    0    0    0    1
## [3,] 3.001704    1    0    0    0    0    0    0    0    1
## [4,] 3.288825    1    0    0    0    0    0    0    0    1
## [5,] 2.905753    1    0    0    0    0    0    0    0    1
## [6,] 2.493667    1    0    0    0    0    0    1    0    0
```

Reference-level coding

Reference-level coding is similar to one-hot encoding except one of the levels of the attribute, called the **reference level**, is omitted. Notice that the 4 dummy columns from Table 1.1 collectively form a linearly dependent set; that is, if you know the values of 3 of the 4 dummy variables you can determine the 4th with complete certainty. This would be a problem for linear regression, where we assume our input attributes are not linearly dependent as we will discuss when we get to Simple Linear Regression.

A reference level of the attribute is often specified by the user to be a particular level worthy of comparison (a baseline), as the estimates in the regression output will be interpreted in a way that compares each non-reference level to the reference level. If a reference level is not specified by the user, one will be picked by the software by default either using the order in which the levels were encountered in the data, or their alphabetical ordering. Users should check the documentation of the associated function to understand what to expect.

Table 1.2 transforms the one-hot encoding from Table 1.1 into reference-level coding with the color “blue” as the reference level. Notice the absence of the column indicating “blue” and how each blue observation exists as a row of zeros.

Observation	Color	Red	Yellow	Other
1	Blue	0	0	0
2	Yellow	0	1	0
3	Blue	0	0	0
4	Red	1	0	0
5	Red	1	0	0
6	Blue	0	0	0
7	Yellow	0	1	0
8	Other	0	0	1

Table 1.2: Reference-level dummy variable coding for the categorical attribute *color* and the reference level of “blue”

Effects coding

Effects coding is useful for obtaining a more general comparative interpretation when you have approximately equal sample sizes across each level of your categorical attribute. Effects coding is designed to allow the user to compare each level to *all* of the other levels. More specifically the mean of each level is compared to the **overall mean** of your data. However,

the comparison is actually to the so-called *grand mean*, which is the mean of the means of each group. When sample sizes are equal, the grand mean and the overall sample mean are equivalent. When sample sizes are *not* equal, the parameter estimates for effects coding should not be used for interpretation or explanation.

Effects coding still requires a *reference level*, however the purpose of the reference level is not the same as it was in reference-level coding. Here, the reference level is left out in the sense that no comparison is made between it and the overall mean. Table 1.3 shows our same example with effects coding. Again we notice the absence of the column indicating “blue” but now the reference level receives values of -1 rather than 0 for all 3 dummy columns. We will revisit the interpretation of linear regression coefficients under this coding scheme in the Simple Linear Regression Section.

Observation	Color	Red	Yellow	Other
1	Blue	-1	-1	-1
2	Yellow	0	1	0
3	Blue	-1	-1	-1
4	Red	1	0	0
5	Red	1	0	0
6	Blue	-1	-1	-1
7	Yellow	0	1	0
8	Other	0	0	1

Table 1.3: Effects coding for the categorical attribute *color* and the reference level of “blue”

1.1.1.2 Interval Variables

An interval variable is a *quantity of interest* on which the mathematical operations of addition, subtraction, multiplication and division can be performed. Time, temperature and age are all examples of interval attributes. To illustrate the definition, note that “15 minutes” divided by “5 minutes” is 3, which indicates that 15 minutes is 3 times as long as 5 minutes. The sensible interpretation of this simple arithmetic sentence demonstrates the nature of interval attributes. One should note that such arithmetic would not make sense in the treatment of nominal variables.

1.1.1.3 Ordinal Variables

Ordinal variables are attributes that are qualitative in nature but have some natural ordering. *Level of education* is a common example, with a level of ‘PhD’ indicating *more* education than ‘Bachelors’ but lacking a numerical framework to quantify *how much more*. The treatment of ordinal variables will depend on the application. Survey responses on a Likert scale are also ordinal - a response of 4=“somewhat agree” on a 1-to-5 scale of agreement cannot reliably be said to be twice as enthusiastic as a response of 2=“somewhat disagree”. These are not interval measurements, though they are often treated as such in a trade-off for computational efficiency.

Ordinal variables will either be given some numeric value and treated as interval variables or they will be treated as categorical variables and dummy variables will be created. The choice of solution is up to the analyst. When numeric values are assigned to ordinal variables, the possibilities are many. For example, consider *level of education*. The simplest ordinal treatment for such an attribute might be something like Table 1.4.

Level of Education	Numeric Value
No H.S. Diploma	1
H.S. Diploma or GED	2
Associates or Certificate	3
Bachelors	4
Masters	5
PhD	6

Table 1.4: One potential approach to scaling the ordinal attribute level of education

While numeric values have been assigned and this data *could* be used like an interval attribute, it's important to realize that the notion of a “one-unit increase” is qualitative in nature rather than quantitative. However, if we're interested in learning whether there is a *linear* type of relationship between education and another attribute (meaning as education level increases, the value of another attribute increases or decreases), this would be the path to get us there. However we're making an assumption in this model that the difference between a H.S. Diploma and an Associates degree (a difference of “1 unit”) is the same as the difference between a Master's degree and a PhD (also a difference of “1 unit”). These types of assumptions can be flawed, and it is often desirable to develop an alternative system of measurement based either on domain expertise or the target variable of interest. This is the notion behind **optimal scaling** and **target-level encoding**.

Optimal Scaling

The primary idea behind optimal scaling is to transform an ordinal attribute into an interval one in a way that doesn't restrict the numeric values to simply the integers 1, 2, 3, . . . It's reasonable for a data scientist to use domain expertise to develop an alternative scheme.

For example, if analyzing movie theater concessions with ordinal drink sizes {small, medium, large}, one is not restricted to the numeric valuation of 1=small, 2=medium, and 3=large just because it's an ordinal variable with 3 levels. Perhaps it would make more sense to use the

drink size in fluid ounces to represent the ordinality. If the small drink is 12 ounces, the medium is 20 ounces, and the large is 48 ounces, then using those values as the numerical representation would be every bit as (if not more) reasonable than using the standard integers 1, 2, and 3.

If we re-consider the ordinal attribute level of education, we might decide to represent the approximate years of post-secondary schooling required to obtain a given level. This might lead us to something like the attribute values in Table ??.

Level of Education	Numeric Value
No H.S. Diploma	8
H.S. Diploma or GED	13
Associates or Certificate	15
Bachelors	17
Masters	19
PhD	22

Table 1.4: One potential approach to scaling the ordinal attribute level of education

If we were modeling the effect of education on something like salary, it seems reasonable to assume that the jumps between levels should not have equal distance like they did in 1.4. It seems reasonable to assume that one would experience a larger salary lift from Associate’s to Bachelor’s degree than they would from No H.S. Diploma to H.S Diploma. The most common way to determine the numeric values for categories is to use information from the response variable. This is commonly referred to as **target level encoding**.

Target Level Encoding

The values in Table ?? might have struck the reader as logical but arbitrary. To be more scientific about the determination of those numeric values, one might wish to use information from the response variable to obtain a more precise expected change in salary for each level

increase in education. At first hearing this, one might question the validity of the technique; isn't the goal to *predict* salary? This line of thought is natural, which is why having a holdout sample is extremely important in this situation. To implement Target level encoding, we can simply create a look-up table that matches each level of education to the average or median salary obtained for that level. These values can be used just as readily as the arbitrary levels created in Table ?? to encode the ordinal attribute!

1.1.2 Distributions

After reviewing the types and formats of the data inputs, we move on to some basic **univariate** (one variable at a time) analysis. We start by describing the distribution of values that each variable takes on. For nominal variables, this amounts to frequency tables and bar charts of how often each level of the variable appears in the data set.

We'll begin by exploring one of our nominal features, `Heating_QC` which categorizes the quality and condition of a home's heating system. To create plots in R, we will use the popular `ggplot2` library. At the same time, we will load the `tidyverse` library which we will use in the next chunk of code.

```
library(ggplot2)
library(tidyverse)
ggplot(data = ames) +
  geom_bar(mapping = aes(x = Heating_QC), fill="orange") + labs(x="Heating System
```

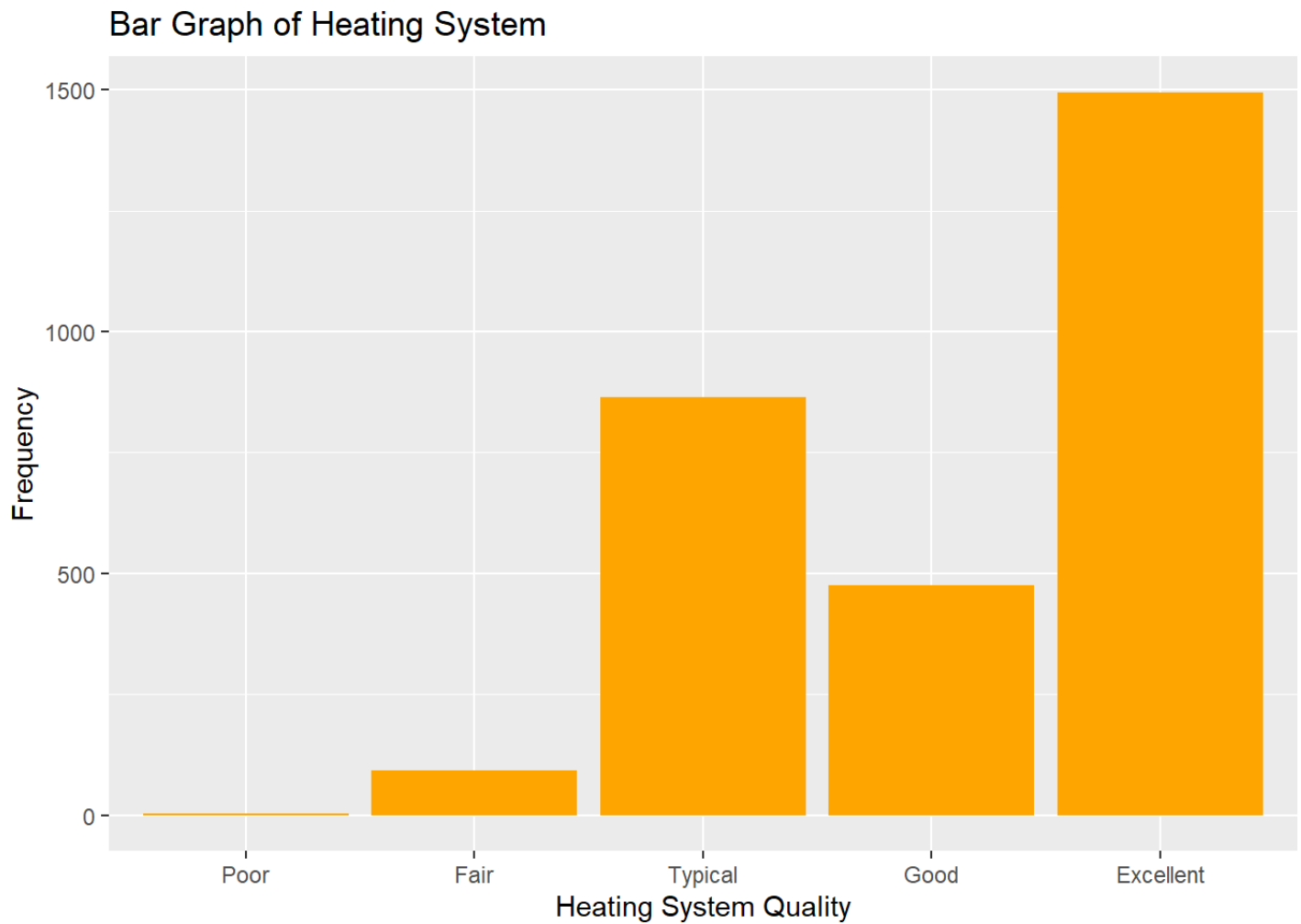


Figure 1.1: Distribution of Nominal Variable Heating_QC

To summon the same information in tabular form, we can use the `count()` function to create a table:

```
ames %>%  
  count(Heating_QC)
```

```
## # A tibble: 5 × 2
##   Heating_QC      n
##   <ord>      <int>
## 1 Poor          3
## 2 Fair         92
## 3 Typical     864
## 4 Good       476
## 5 Excellent  1495
```

You'll notice that very few houses (3) have heating systems in `Poor` condition, and the majority of houses have systems rated `Excellent` . **It will likely make sense to combine the categories of `Fair` and `Poor` in our eventual analysis, a decision we will later revisit.**

Next we create a **histogram** for an interval attribute like `Sale_Price` :

```
ggplot(data = ames) +
  geom_histogram(mapping = aes(x = Sale_Price/1000), fill="blue") +
  labs(x = "Sales Price (Thousands $)", y="Frequency", title="Histogram of Sales Pi
```

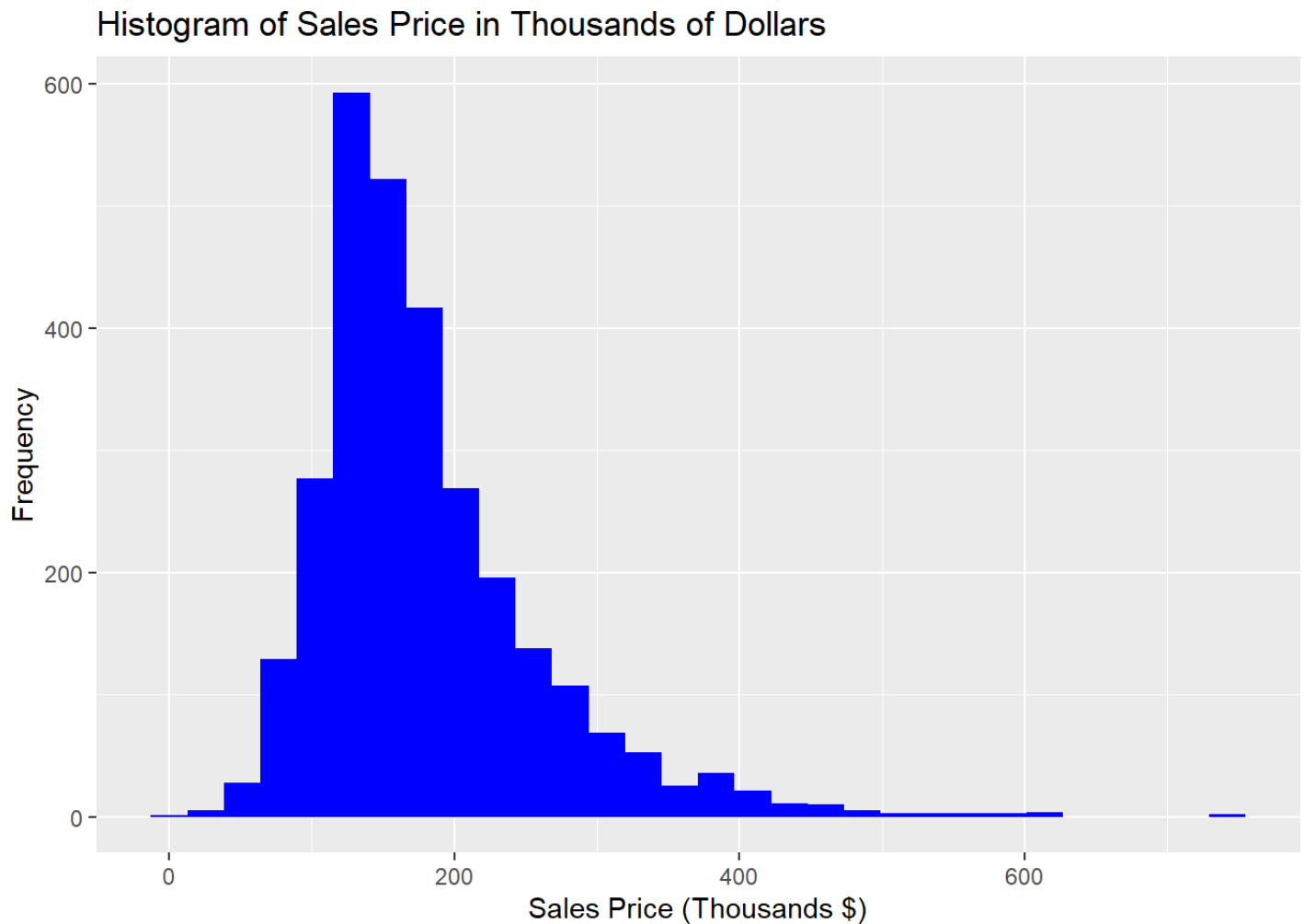



Figure 1.2: Distribution of Interval Variable Sale_Price

From this initial inspection, we can conclude that most of the houses sell for less than \$200,000 and there are a number of expensive anomalies. To more concretely describe and quantify a statistical distribution, we use statistics that describe the *location*, *spread*, and *shape* of the data.

1.1.2.1 Location

The *location* is a measure of central tendency for a distribution. Most common measures of central tendency are **mean**, **median**, and **mode**.

We define each of these terms below for a variable $\mathbf{x}$ having n observations with values $\{x_i\}_{i=1}^n$, sorted in order of magnitude such that $x_1 \leq x_2 \leq \dots \leq x_n$:

- Mean: The **average** of the observations, $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
- Median: The “middle value” of the data. Formally, when n is odd, the median is the observation value, $x_m = x_{\frac{(n+1)}{2}}$ for which $x_i < x_m$ for 50% of the observations (excluding x_m). When n is even, x_m is the average of $x_{\frac{n}{2}}$ and $x_{(\frac{n}{2}+1)}$. The median is also known as the **2nd quartile** (see next section).
- Mode: The most commonly occurring value in the data. Most commonly used to describe nominal attributes.

Example

The following table contains the heights of 10 students randomly sampled from NC State’s campus. Compute the mean, median, mode and quartiles of this variable.

height	60	62	63	65	67	67	67	68	68	69
--------	----	----	----	----	----	----	----	----	----	----

Solution:

- The mean is $(60+62+63+65+67+67+67+68+68+69)/10 = 65.6$.
- The median (second quartile) is $(67+67)/2 = 67$.
- The mode is 67.

1.1.2.2 Spread

Once we have an understanding of the central tendency of a data set, we move on to describing the spread (the dispersion or variation) of the data. **Range**, **interquartile range**, **variance**, and **standard deviation** are all statistics that describe spread.

- Range: The difference between the maximum and minimum data values.
- Sample variance: The sum of squared differences between each data point and the mean, divided by $(n-1)$. $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$
- Standard deviation: The square root of the sample variance.

In order to define **interquartile range**, we must first define **percentiles** and **quartiles**.

- **Percentiles:** The 99 intermediate values of the data which divide the observations into 100 equally-sized groups. The r^{th} percentile of the data, P_r is the number for which $r\%$ of the data is less than P_r . Percentiles are also called Quantiles.
- **Quartiles:** The quartiles of the data are the 25^{th} , 50^{th} and 75^{th} percentiles. They are denoted as Q_1 (1^{st} quartile), Q_2 (2^{nd} quartile = median) and Q_3 (3^{rd} quartile), respectively.
- **Interquartile range (IQR):** The difference between the 25^{th} and 75^{th} percentiles.

One should note that standard deviation is more frequently reported than variance because it shares the same units as the original data, and because of the guidance provided by the empirical rule. If we're exploring something like `Sale_Price`, which has the unit “dollars”, then the variance would be measured in “square-dollars”, which hampers the intuition. Standard deviation, on the other hand, would share the unit “dollars”, aiding our fundamental understanding.

Example Let's again use the table of heights from the previous example, this time computing the range, IQR, sample variance and standard deviation.

height	60	62	63	65	67	67	67	68	68	69
--------	----	----	----	----	----	----	----	----	----	----

Solution:

- The range $69 - 60 = 9$.
- The variance is $((60 - 65.6)^2 + (62 - 65.6)^2 + (63 - 65.6)^2 + (65 - 65.6)^2 + (67 - 65.6)^2 + (67 - 65.6)^2 + (67 - 65.6)^2 + (68 - 65.6)^2 + (68 - 65.6)^2 + (69 - 65.6)^2) / 9 = 8.933$
- The standard deviation is $\sqrt{8.933} = 2.989$
- The first quartile is 63.5
- The third quartile is 67.75
- The IQR is $68 - 62.5 = 4.25$.

Note: In R, to find the quartiles, you should ask for the 0.25, 0.5 and 0.75 quantiles.

1.1.2.3 Shape

The final description we will want to give to distributions regards their shape. Is the histogram *symmetric*? Is it *unimodal* (having a single large “heap” of data) or *multimodal* (having multiple heaps)? Does it have a longer tail on one side than the other (*skew*)? Is there a lot more or less data in the tails than you might expect?

We’ll formalize these ideas with some illustrations. A distribution is right (left) skewed if it has a longer tail on its right (left) side, as shown in Figure 1.3.

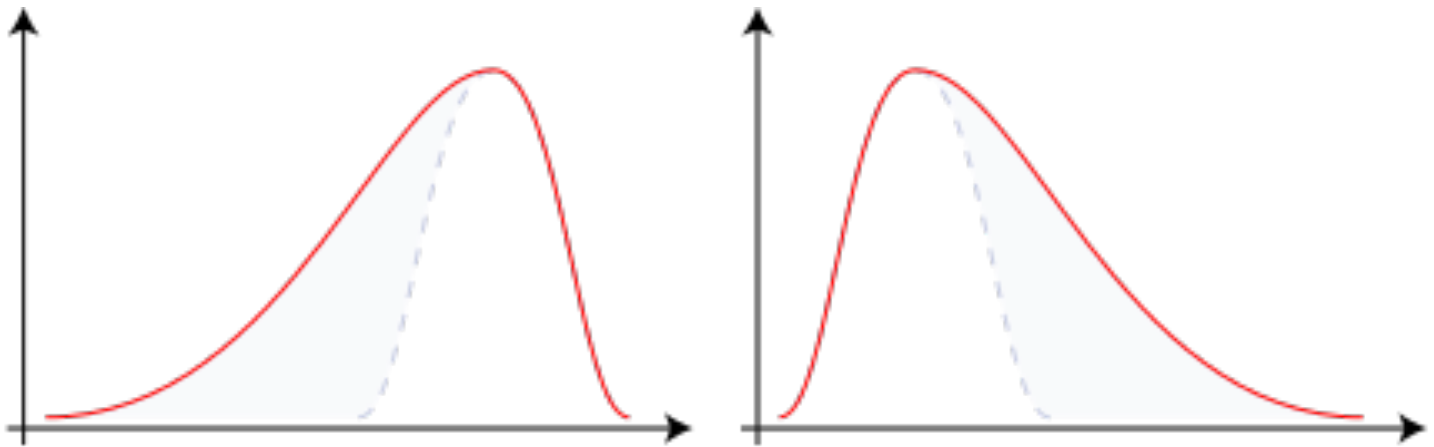


Figure 1.3: Examples of Left-Skewed (Negative Skew) and Right-skewed (Positive Skew) distributions respectively

A distribution is called *bimodal* if it has two “heaps”, as shown in Figure 1.4.

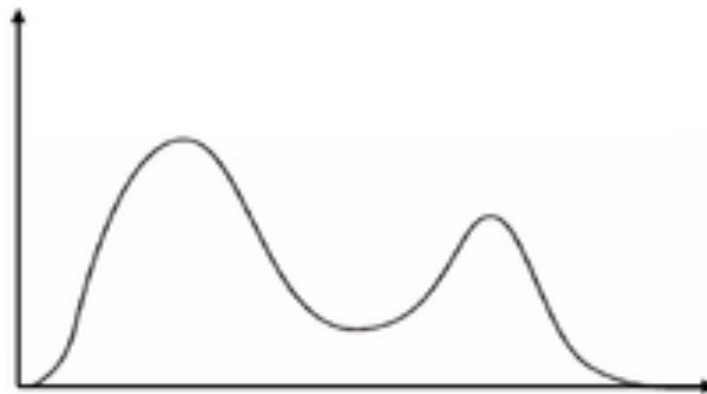


Figure 1.4: Example of a Bimodal Distribution

1.1.2.4 Summary Functions in R

There are many ways to obtain all of the statistics described in the preceding sections, below we highlight 3:

- The `describe` function from the `Hmisc` package which can work on the entire dataset or a subset of columns.

```
library(Hmisc)
```

```
Hmisc::describe(ames$Sale_Price)
```

```
## ames$Sale_Price
##           n missing distinct      Info      Mean      Gmd       .05       .10
##      2930         0      1032         1  180796    81960    87500    105450
##       .25       .50       .75       .90       .95
##  129500    160000    213500    281242    335000
##
## lowest :  12789  13100  34900  35000  35311, highest: 611657 615000 625000 74!
```

- The tidyverse `summarise` function, in this case obtaining statistics for each `Exter_Qual` separately.

```
library(tidyverse)
```

```
ames %>% group_by(Exter_Qual) %>% dplyr::summarise(average = mean(Sale_Price), si
```

```
## # A tibble: 4 × 5
##   Exter_Qual average  st.dev maximum minimum
##   <ord>         <dbl>   <dbl>   <int>   <int>
## 1 Fair          89924.   38014.   200000   13100
## 2 Typical      143374.   41504.   415000   12789
## 3 Good         230756.   70411.   745000   52000
## 4 Excellent   377919.  106988.   755000  160000
```

- The base R `summary` function, which can work on the entire dataset or an individual variable

```
summary(ames$Sale_Price)
```

```
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  12789  129500  160000  180796  213500  755000
```

1.1.3 The Normal Distribution

The normal distribution, also known as the Gaussian distribution, is one of the most fundamental concepts in statistics. It is one that arises naturally out of many applications and settings. The Normal distribution has the following characteristics:

- Symmetric
- Fully defined by mean and standard deviation (equivalently, variance)
- Bell-shaped/Unimodal
- Mean = Median = Mode
- Asymptotic to the x-axis (theoretical bounds are $-\infty$ to ∞)

Much of the normal distributions utility can be summarized in the **empirical rule**, which states that:

- $\approx 68\%$ of data in normal distribution lies within 1 standard deviation of the mean.
- $\approx 95\%$ of data in normal distribution lies within 2 standard deviations of the mean.
- $\approx 99.7\%$ of data in normal distribution lies within 3 standard deviations of the mean.

We can thus conclude that observations found outside of 3 standard deviations from the mean are quite rare, expected less than 1% of the time.

1.1.4 Skewness

Skewness is a statistic that describes the symmetry (or lack thereof) of a distribution. A normal distribution is perfectly symmetric and has a skewness of 0. Distributions that are more right skewed will have positive values of skewness whereas distributions that are more left skewed will have negative values of skewness.

1.1.5 Kurtosis

Kurtosis is a statistic that describes the *tailedness* of a distribution. The normal distribution has a kurtosis of 3. Distributions that are more tailed (leptokurtic or heavy-tailed) will have kurtosis values greater than 3 whereas distributions that are more less tailed (platykurtic or thin-tailed) will have values of kurtosis less than 3. For this reason, kurtosis is often reported in the form of *excess kurtosis* which is the raw kurtosis value minus 3. This is meant as a comparison to the normal distribution so that positive values indicate thicker tails and negative values indicate thinner tails than the normal.

In Figure 1.5 below, we compare classical examples of leptokurtic and platykurtic distributions to a normal distribution with the same mean and variance.

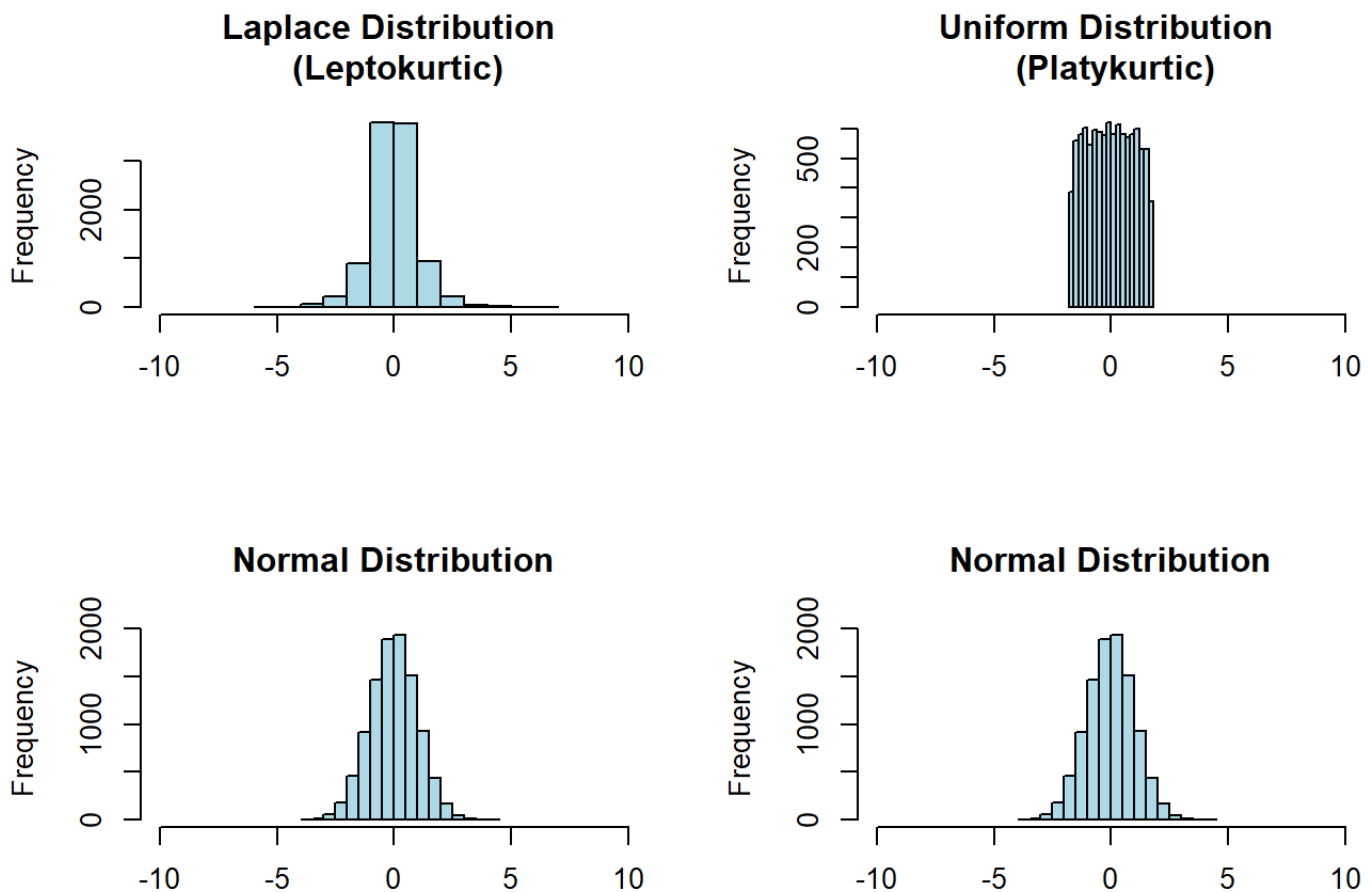


Figure 1.5: The Laplace distribution (top left) is leptokurtic because it has more data in its tails than the normal distribution with the same mean and variance. The uniform distribution (top right) is platykurtic because it has less data in its tails than the normal distribution with the same mean and variance (it effectively has no tails).

1.1.6 Graphical Displays of Distributions

Three common plots for examining the distribution of your data values:

1. Histograms
2. Normal Probability Plots (QQ-plots)
3. Box Plots

1.1.6.1 Histograms

A histogram shows the shape of a univariate distribution. Each bar in the histogram represents a group of values (a *bin*). The height of the bar represents either the frequency of or the percent of values in the bin. The width and number of bins is determined automatically, but the user can adjust them to see more or less detail in the histogram. Figure 1.2 demonstrated a histogram of sale price. Sometimes it's nice to overlay a continuous approximation to the underlying distribution using a *kernal density estimator* with the `geom_density` plot, demonstrated in Figure 1.6.

```
ggplot(ames, aes(x=Sale_Price/1000)) +  
  geom_histogram(aes(y=..density..), alpha=0.5) + geom_density(alpha = 0.2) +  
  labs(x = "Sales Price (Thousands $)")
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.  
## i Please use `after_stat(density)` instead.  
## This warning is displayed once every 8 hours.  
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was  
## generated.
```

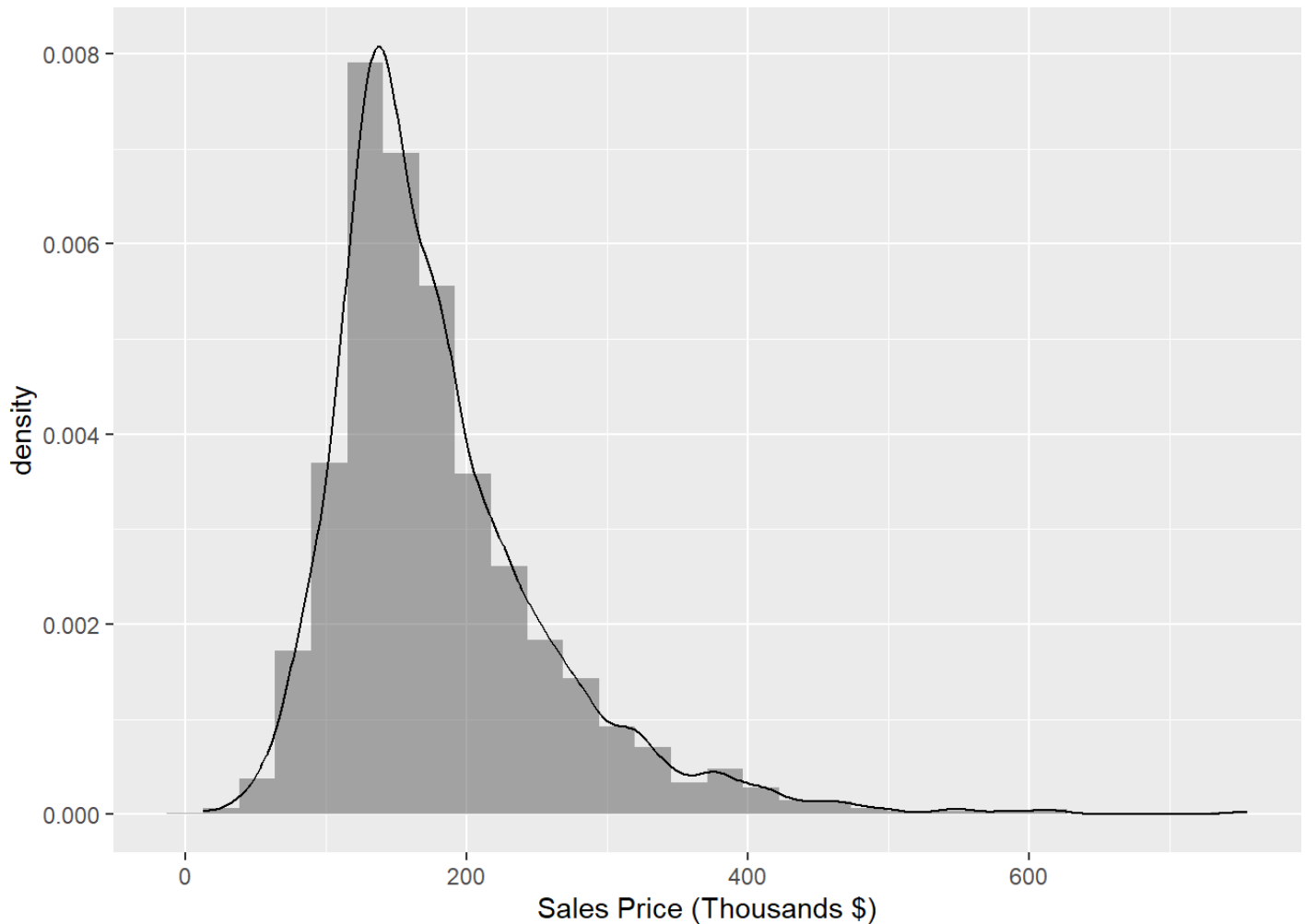


Figure 1.6: Histogram of Sale_Price with kernel density estimator

In our next example, Figure 1.7, we'll complicate the previous example by showing *two* distributions of sale price, one for each level of the binary variable `Central_Air`, overlaid on the same axes.

We can immediately see that there are many more houses that have central air than do not in this data. It appears as though the two distributions have different locations, with the purple distribution centered at a larger sale price. To normalize that quantity and compare the raw probability densities, we can change our axes to density as in Figure 1.8.

```
ggplot(ames, aes(x=Sale_Price/1000)) +
  geom_histogram(data=subset(ames, Central_Air == 'Y'), aes(fill=Central_Air), a
  geom_histogram(data=subset(ames, Central_Air == 'N'), aes(fill=Central_Air), a
  labs(x = "Sales Price (Thousands $)") + scale_fill_manual(name="Central_Air"
```

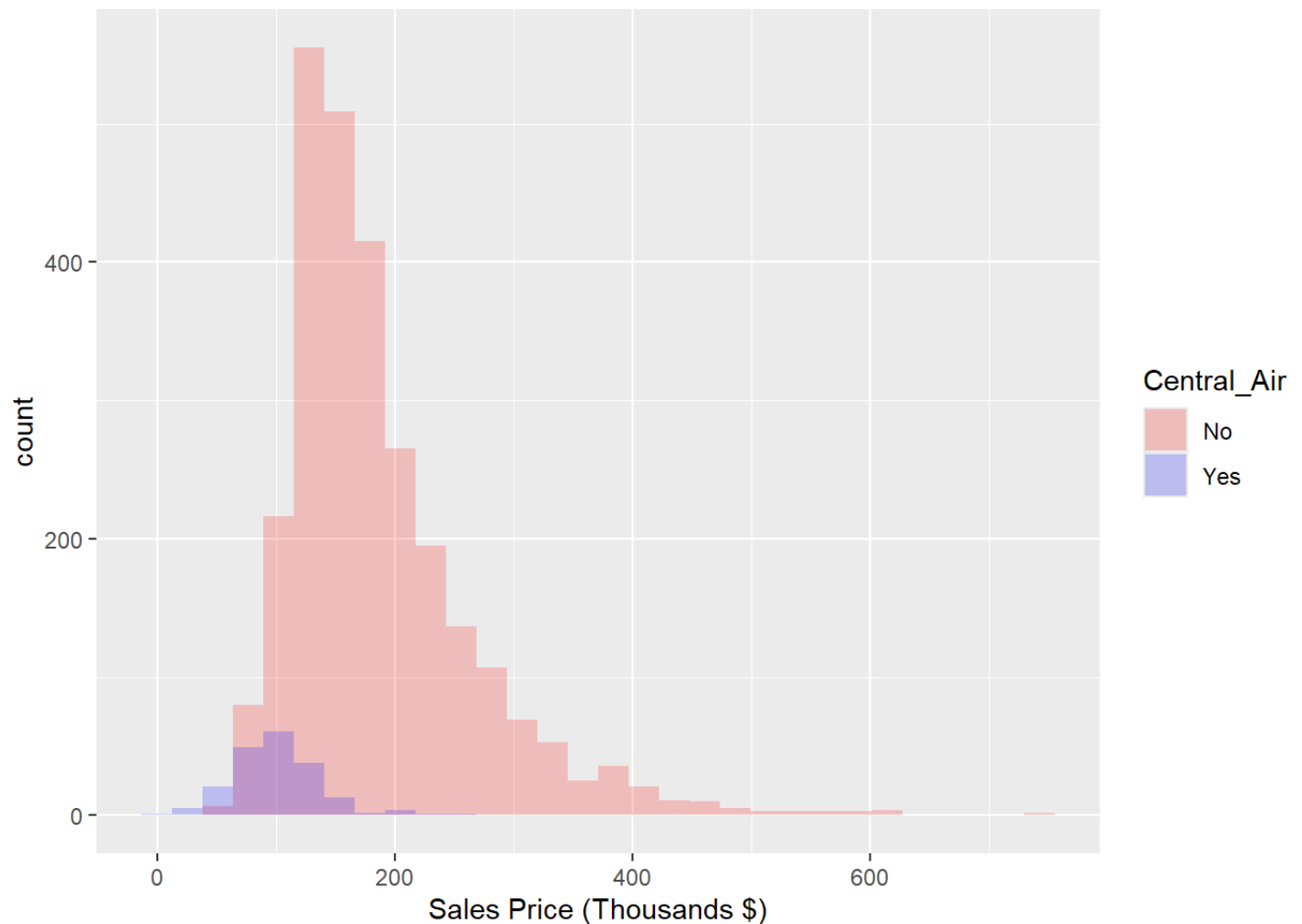


Figure 1.7: Histogram: Frequency of Sale_Price for Each value of Central_Air

```
ggplot(ames, aes(x=Sale_Price/1000)) +
  geom_histogram(data=subset(ames, Central_Air == 'Y'), aes(y=..density.., fill=Ce
  geom_histogram(data=subset(ames, Central_Air == 'N'), aes(y=..density.., fill=Ce
  labs(x = "Sales Price (Thousands $)")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.  
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

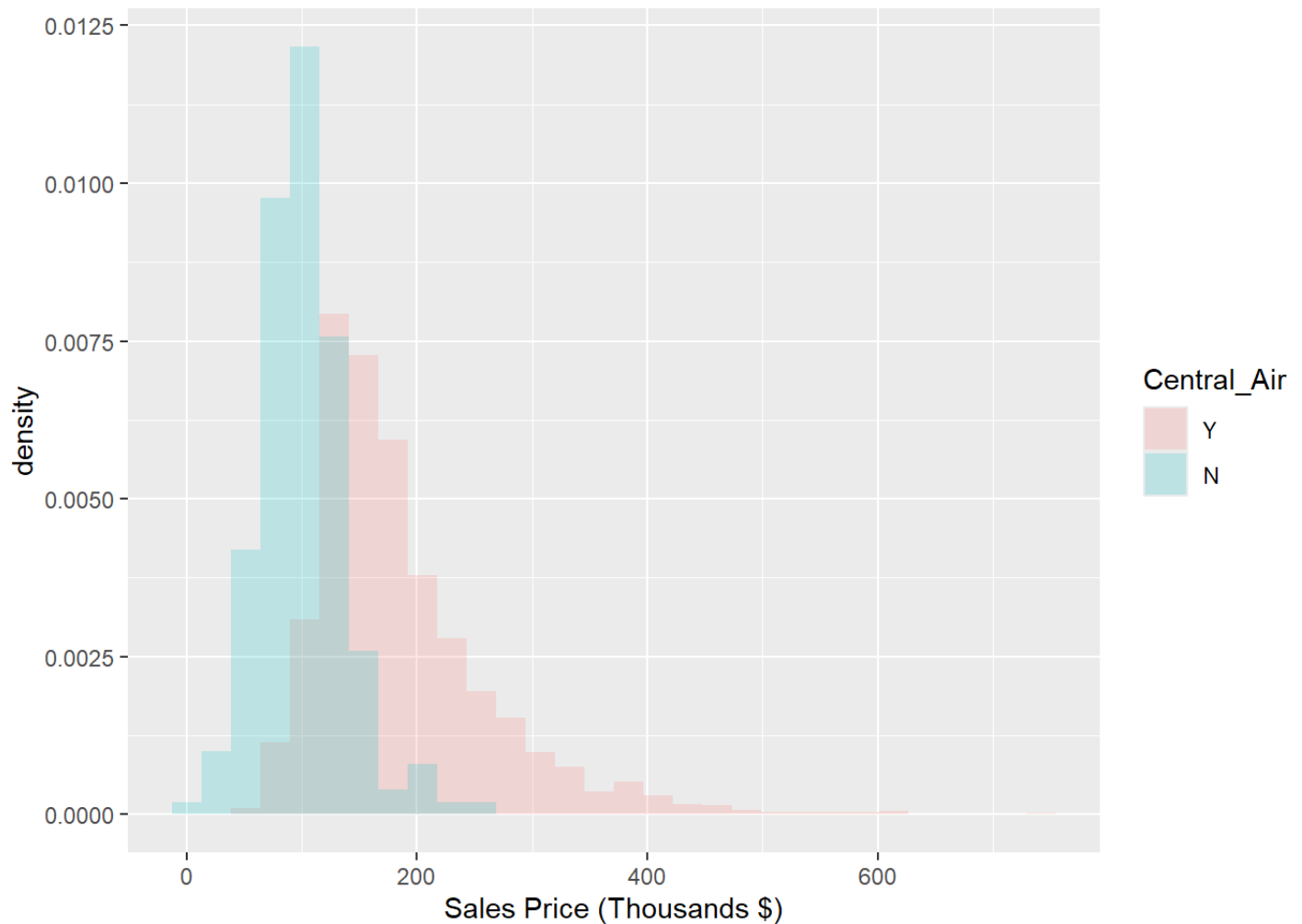


Figure 1.8: Histogram: Density of Sale_Price for varying qualities of Central_Air

To ease our differentiation of the histograms even further, we again employ a kernel density estimator as shown in Figure 1.9. This is an appealing alternative to the histogram for continuous data that is assumed to originate from some smooth underlying distribution.

```
ggplot(ames, aes(x=Sale_Price/1000)) +  
  geom_density(data=subset(ames, Central_Air == 'Y'), aes(fill=Central_Air), alpha=0.5)  
  geom_density(data=subset(ames, Central_Air == 'N'), aes(fill=Central_Air), alpha=0.5)  
  labs(x = "Sales Price (Thousands $)")
```

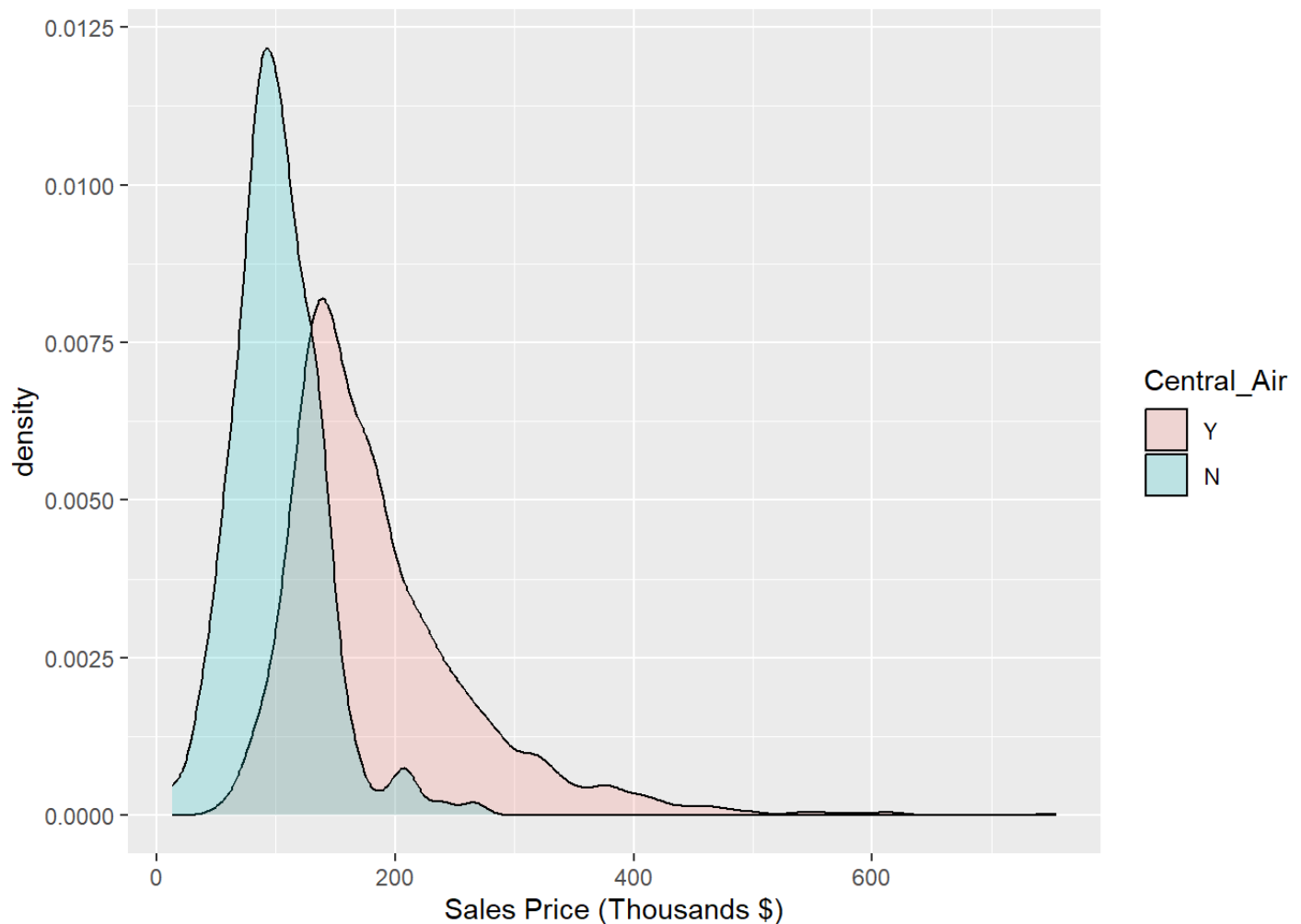


Figure 1.9: Histogram: Density of Sale_Price for varying qualities of Central_Air

1.1.6.2 Normal probability plots (QQ Plots)

A **normal probability plot** (or QQ plot) graphs the sorted data values against the values that one would expect if the same number of observations came from a theoretical Normal distribution. The resulting image would look close to a straight line if the data was generated

by a Normal distribution. Strong deviations from a straight line indicate that the data distribution is not Normal.

Figure 1.10 shows a QQ plot for `Sale_Price`, and we can conclude that the variable is not Normally distributed (in fact it is right skewed).

```
ggplot(data = ames, aes(sample = Sale_Price/1000)) +  
  stat_qq() +  
  stat_qq_line()
```

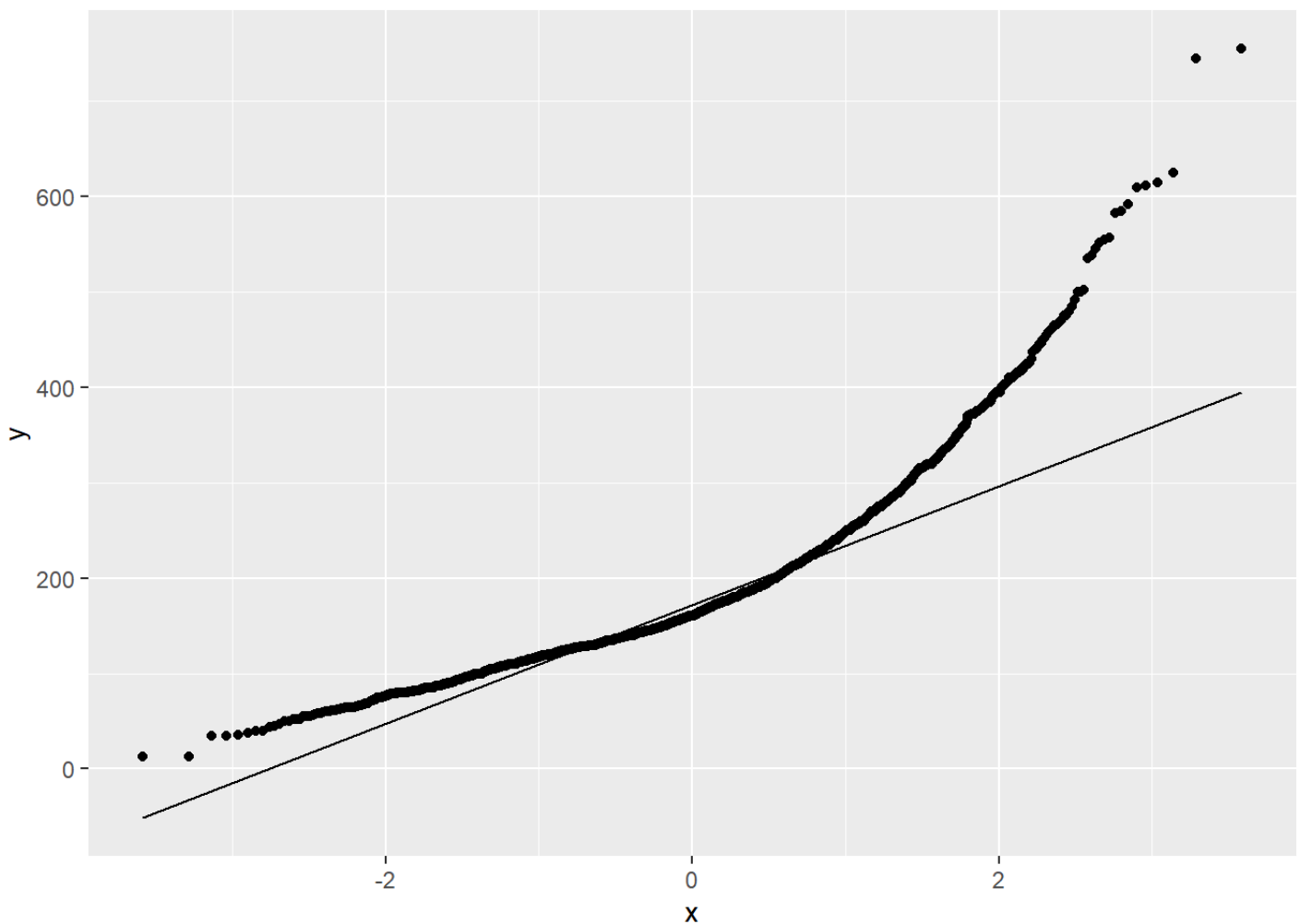


Figure 1.10: QQ-Plot: Quantiles of `Sale_Price` vs. quantiles of a theoretical normal distribution with same mean and standard deviation. Conclusion: `Sale_Price` is *not* normally distributed due to a problem with skew.

There are two main patterns that we expect to find when examining QQ-plots:

1. A quadratic shape, as seen in Figure 1.10. This pattern indicates a deviation from normality due to skewness to the data.
2. An S-shape (or cubic shape), as seen in Figure 1.11. This pattern indicates deviation from normality due to kurtosis.

```
df <- data.frame(j1 = rlaplace(10000,0,1))
```

```
ggplot(data = df, aes(sample=j1)) +  
  stat_qq() +  
  stat_qq_line()
```

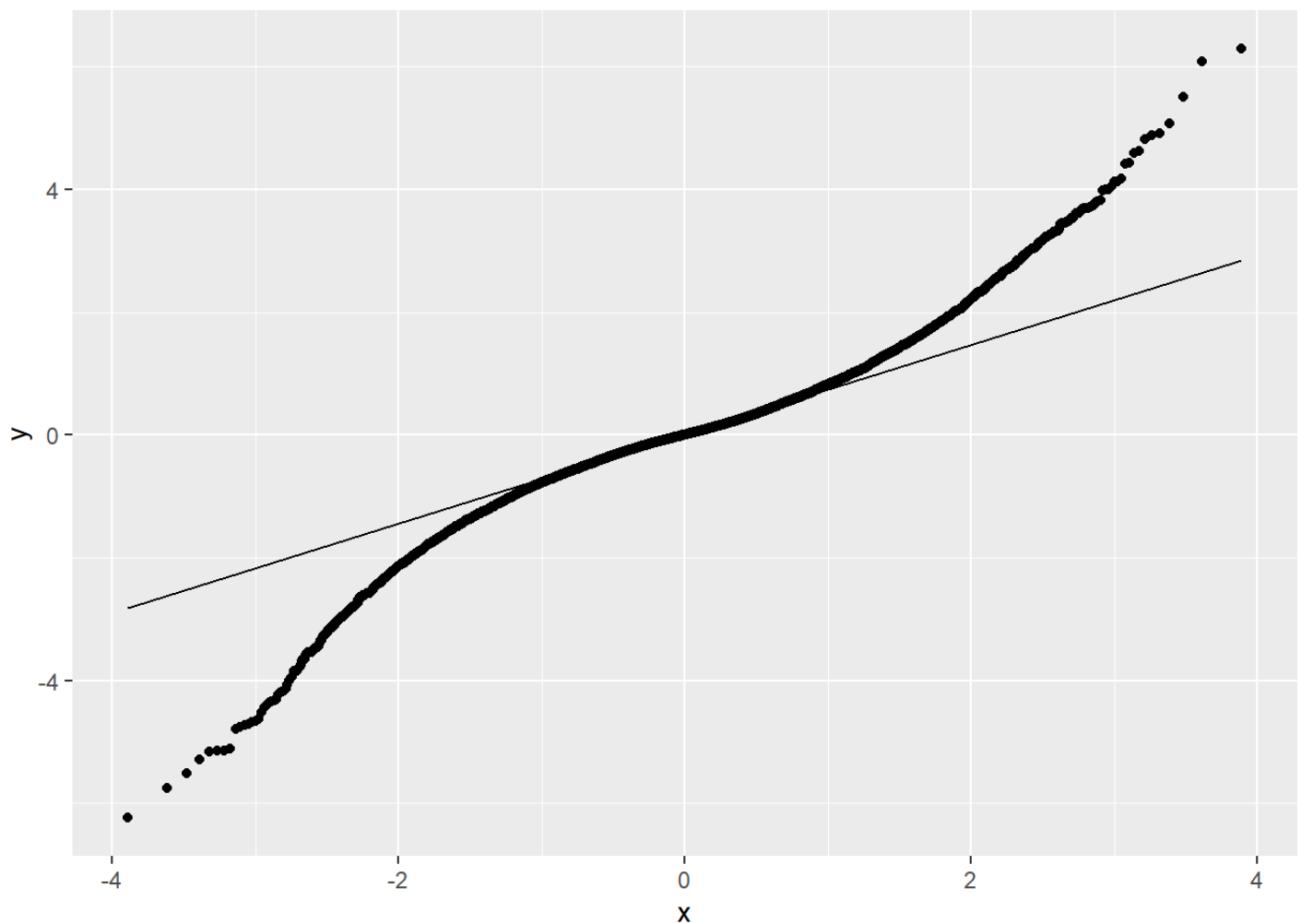


Figure 1.11: QQ-Plot: Quantiles of the Laplace distribution vs. quantiles of a theoretical normal distribution with same mean and standard deviation. Conclusion: Data is *not* normally distributed (in fact it is leptokurtic), due to a problem with kurtosis.

1.1.6.3 Box Plots

Box plots (sometimes called box-and-whisker plots) will not necessarily tell you about the *shape* of your distribution (for instance a bimodal distribution could have a similar box plot to a unimodal one), but it will give you a sense of the distribution's location and spread and potential skewness.

Many of us have become familiar with the *idea* of a box plot, but when pressed for the specific steps to create one, we realize our familiarity fades. The diagram in Figure 1.12 will remind the reader the precise information conveyed by a box plot.

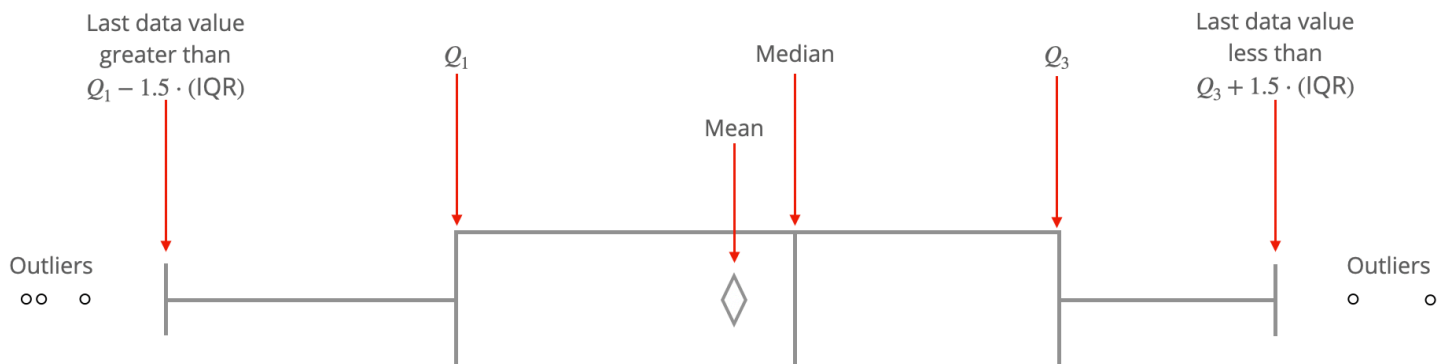


Figure 1.12: Anatomy of a Box Plot.

Figure 1.13 shows the boxplot of `Sale_Price` .

```
ggplot(data = ames, aes(y = Sale_Price/1000)) +
  geom_boxplot() +
  labs(y = "Sales Price (Thousands $)")
```

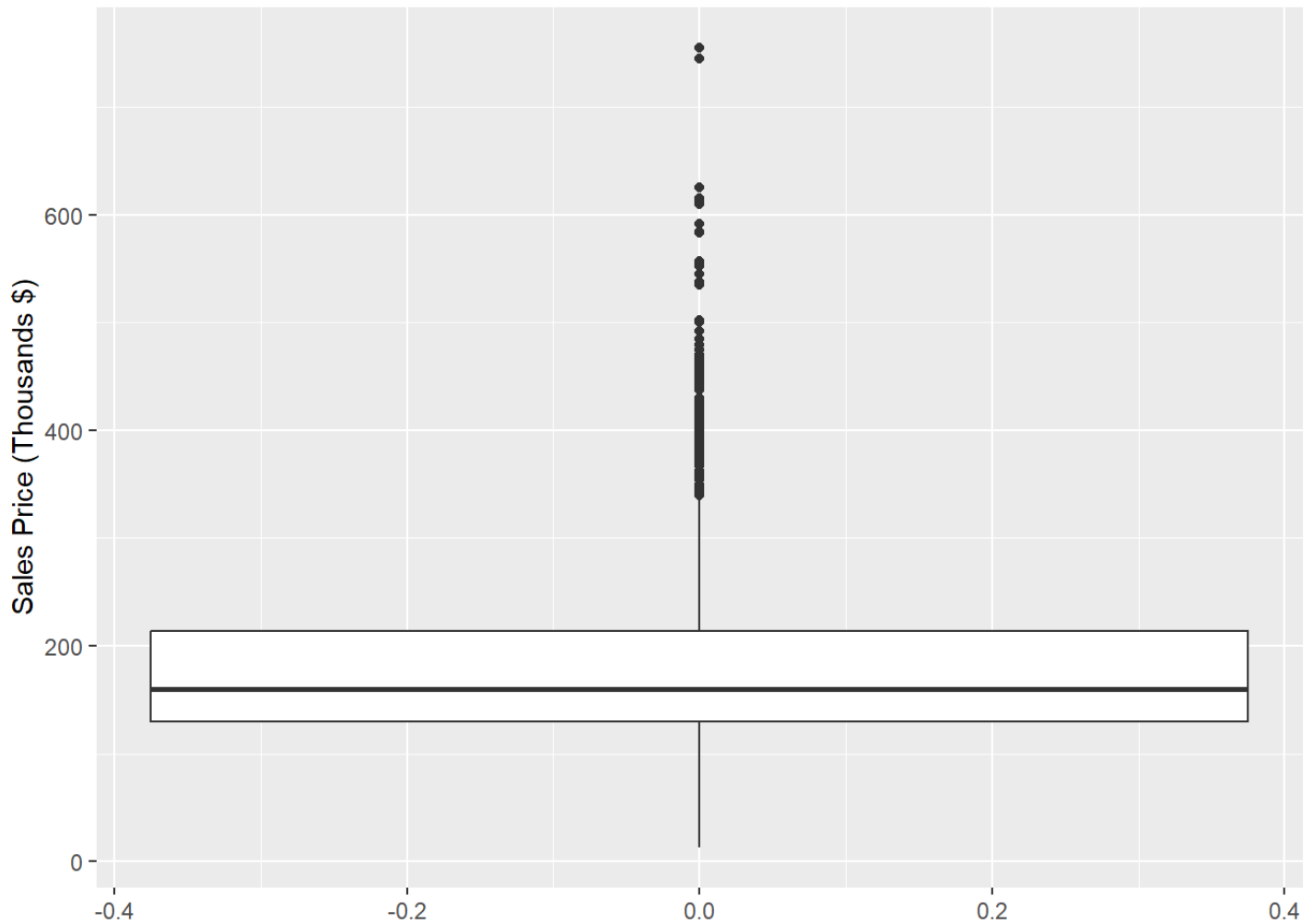



Figure 1.13: Box Plot of Sales Price

Furthermore, we might want to compare the boxplots of `Sale_Price` for different levels of a categorical variable, like `Central_Air` as we did with histograms and densities in Figures 1.7 and 1.8.

The following code achieves this goal in Figure 1.14.

```
ggplot(data = ames, aes(y = Sale_Price/1000, x = Central_Air, fill = Central_Air)) +
  geom_boxplot() +
  labs(y = "Sales Price (Thousands $)", x = "Central Air") +
  scale_fill_brewer(palette="Accent") + theme_classic() + coord_flip()
```

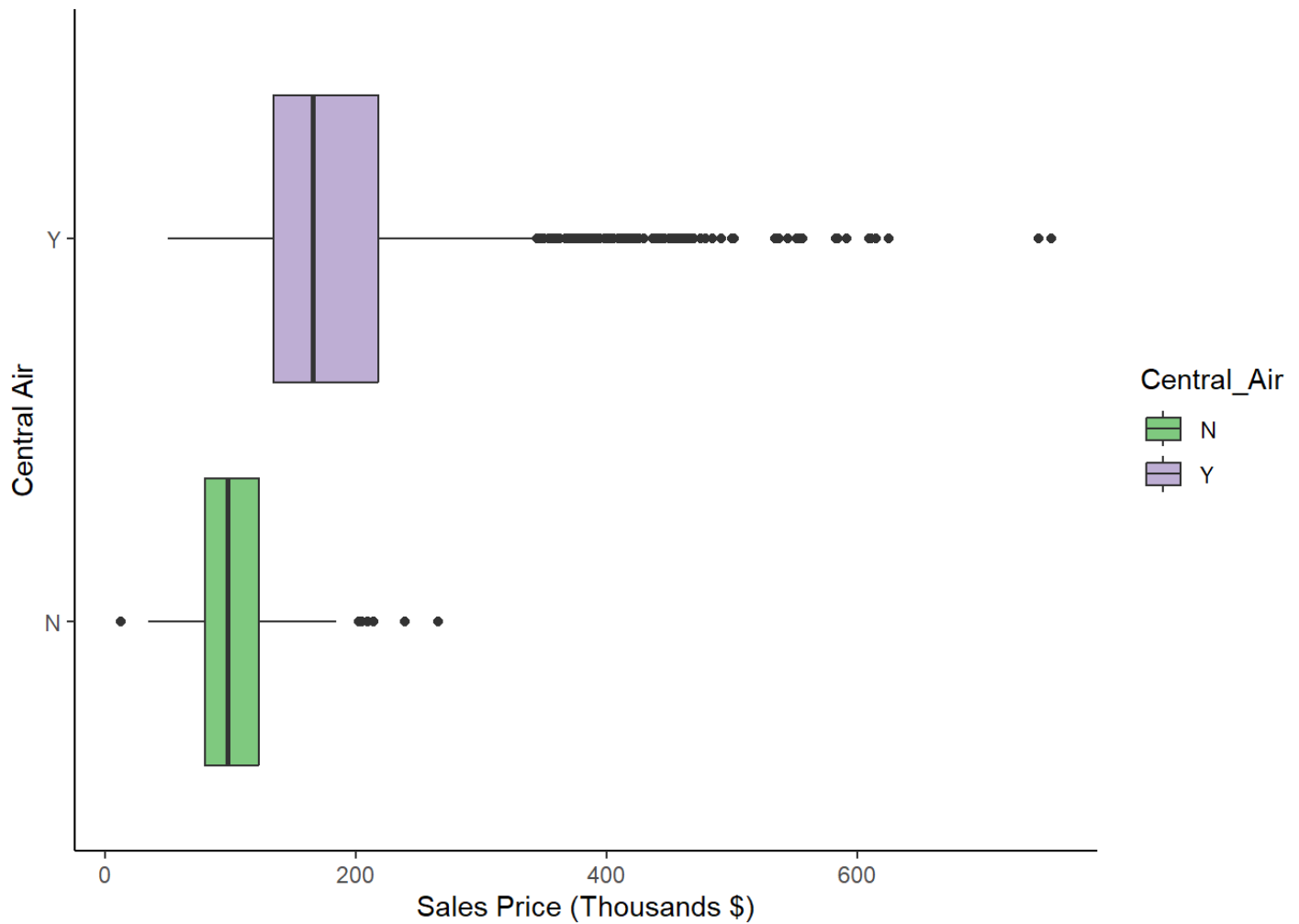


Figure 1.14: Box Plots of Sale_Price for each level of Exter_Qual

1.1.7 Python Code

For all of the Python code throughout the book, you will need to use the following libraries:

```
import pandas as pd
```

```
## C:\Users\sjsimmo2\AppData\Local\R\win-library\4.4\reticulate\python\rpytools\
## return _find_and_load(name, import_)
## C:\Users\sjsimmo2\AppData\Local\R\win-library\4.4\reticulate\python\rpytools\
## return _find_and_load(name, import_)
```

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder
import seaborn as sns
import matplotlib.pyplot as plt
from numpy import random
import statsmodels.api as sma
import statsmodels as sm
import pylab as py
import scipy.stats as stats
import scipy as sp
import statsmodels.formula.api as smf
import sklearn
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_squared_error
```

For one-hot encoding in Python:

```
x1=np.repeat(["A","B","C"],10)
x2=np.repeat(["Z", "X", "Y","W","V","U"],5)
random.seed(41)
y=np.concatenate([np.random.normal(loc=2.0, scale=1.0, size=10),np.random.normal(
array=np.array([x1,x2,y])
array2=np.transpose(array)
column_values=["x1","x2","y"]
df = pd.DataFrame(data = array2,
                  columns = column_values)
print(df)
```

##	x1	x2	y
## 0	A	Z	1.7292876769326795
## 1	A	Z	2.10484805260974
## 2	A	Z	2.250527815723572
## 3	A	Z	1.0748000347219233
## 4	A	Z	2.567143660285906
## 5	A	X	0.9598197839170619
## 6	A	X	1.8463240485420624
## 7	A	X	2.789851810346819
## 8	A	X	0.7737841535581458
## 9	A	X	1.0519930122865415
## 10	B	Y	0.4303460580699353
## 11	B	Y	0.022849785302227588
## 12	B	Y	0.22936828881644922
## 13	B	Y	0.9662887085465139
## 14	B	Y	-0.032859245166130036
## 15	B	W	2.1424273766831528
## 16	B	W	0.3902219923125273
## 17	B	W	2.46941638687667
## 18	B	W	2.4926788367383335
## 19	B	W	1.7071252278892184
## 20	C	V	-1.8584902566516113
## 21	C	V	-1.3706237711281049
## 22	C	V	-0.33010638792060293
## 23	C	V	-1.5152899458722047
## 24	C	V	1.2000601858215505
## 25	C	U	-1.8226191434208407
## 26	C	U	0.26938454136355156
## 27	C	U	-0.44642437564718757
## 28	C	U	1.1143136000921545
## 29	C	U	-1.3808025981774998

```
one_hot_encoded_data = pd.get_dummies(df, columns = ['x1', 'x2'])
print(one_hot_encoded_data)
```

##	y	x1_A	x1_B	x1_C	...	x2_W	x2_X	x2_Y	x2_Z
## 0	1.7292876769326795	True	False	False	...	False	False	False	True
## 1	2.10484805260974	True	False	False	...	False	False	False	True
## 2	2.250527815723572	True	False	False	...	False	False	False	True
## 3	1.0748000347219233	True	False	False	...	False	False	False	True
## 4	2.567143660285906	True	False	False	...	False	False	False	True
## 5	0.9598197839170619	True	False	False	...	False	True	False	False
## 6	1.8463240485420624	True	False	False	...	False	True	False	False
## 7	2.789851810346819	True	False	False	...	False	True	False	False
## 8	0.7737841535581458	True	False	False	...	False	True	False	False
## 9	1.0519930122865415	True	False	False	...	False	True	False	False
## 10	0.4303460580699353	False	True	False	...	False	False	True	False
## 11	0.022849785302227588	False	True	False	...	False	False	True	False
## 12	0.22936828881644922	False	True	False	...	False	False	True	False
## 13	0.9662887085465139	False	True	False	...	False	False	True	False
## 14	-0.032859245166130036	False	True	False	...	False	False	True	False
## 15	2.1424273766831528	False	True	False	...	True	False	False	False
## 16	0.3902219923125273	False	True	False	...	True	False	False	False
## 17	2.46941638687667	False	True	False	...	True	False	False	False
## 18	2.4926788367383335	False	True	False	...	True	False	False	False
## 19	1.7071252278892184	False	True	False	...	True	False	False	False
## 20	-1.8584902566516113	False	False	True	...	False	False	False	False
## 21	-1.3706237711281049	False	False	True	...	False	False	False	False
## 22	-0.33010638792060293	False	False	True	...	False	False	False	False
## 23	-1.5152899458722047	False	False	True	...	False	False	False	False
## 24	1.2000601858215505	False	False	True	...	False	False	False	False

```
## 25    -1.8226191434208407  False  False  True  ...  False  False  False  Fal
## 26     0.26938454136355156  False  False  True  ...  False  False  False  Fal
## 27    -0.44642437564718757  False  False  True  ...  False  False  False  Fal
## 28     1.1143136000921545  False  False  True  ...  False  False  False  Fal
## 29    -1.3808025981774998  False  False  True  ...  False  False  False  Fal
##
## [30 rows x 10 columns]
```

Bring the r data set over to Python (if you are writing this in an R Markdown document).

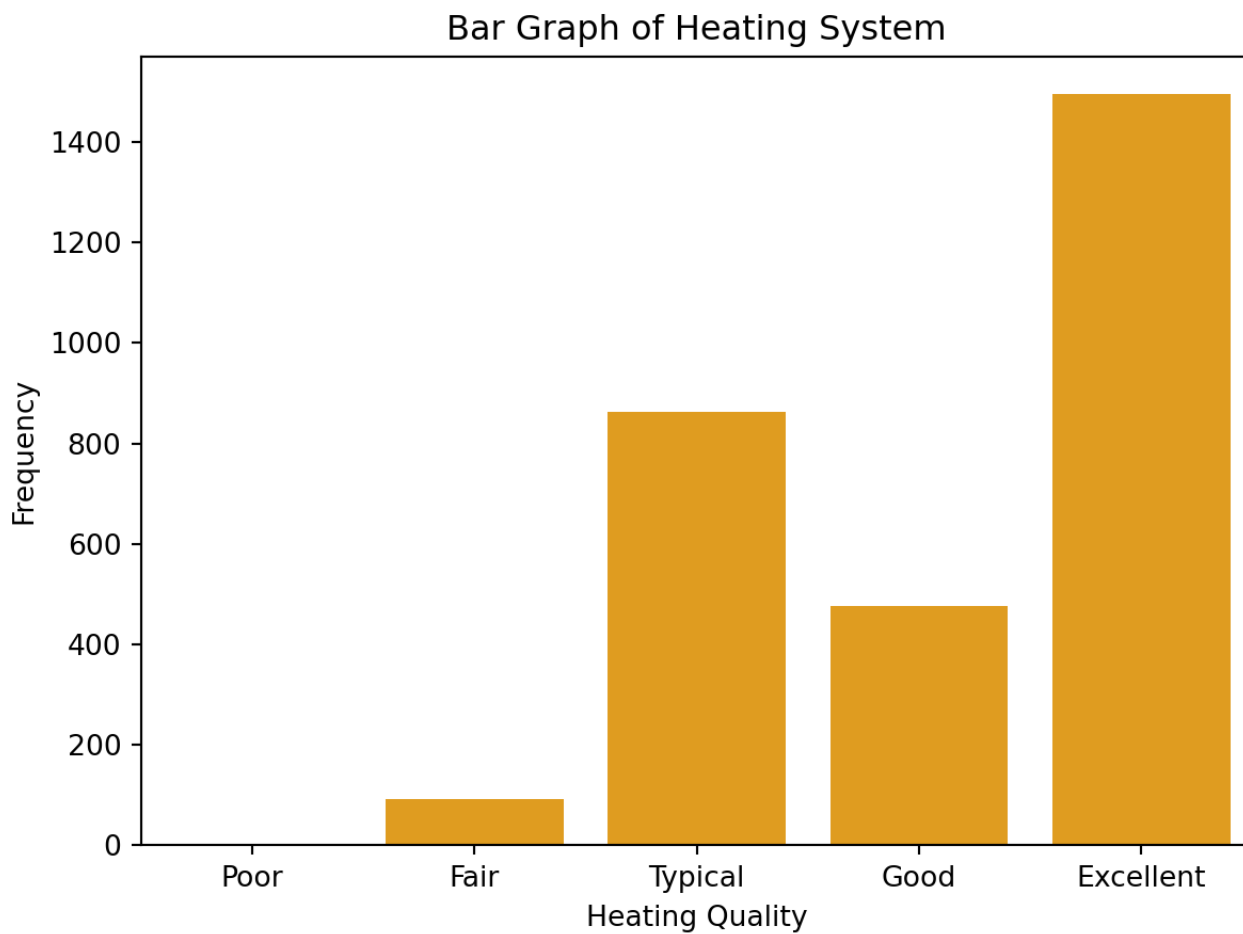
```
ames_py = r.ames
ames_py = pd.DataFrame(ames_py)

ames_py.describe()
```

```
##      Lot_Frontage      Lot_Area  ...      Longitude      Latitude
## count    2930.000000    2930.000000  ...    2930.000000    2930.000000
## mean       57.647782   10147.921843  ...     -93.642897     42.034482
## std        33.499441    7880.017759  ...       0.025700       0.018410
## min         0.000000    1300.000000  ...     -93.693153     41.986498
## 25%         43.000000    7440.250000  ...     -93.660217     42.022088
## 50%         63.000000    9436.500000  ...     -93.641806     42.034662
## 75%         78.000000   11555.250000  ...     -93.622113     42.049853
## max        313.000000   215245.000000  ...     -93.577427     42.063388
##
## [8 rows x 35 columns]
```

Creating a frequency bar plot for Heating Quality:

```
ax = sns.countplot(x = "Heating_QC", data = ames_py, color = "orange")
ax.set(xlabel = 'Heating Quality',
       ylabel = 'Frequency',
       title = 'Bar Graph of Heating System')
plt.show()
```



Creating a table of frequencies for Heating Quality:

```
ames_py['Heating_QC'].value_counts()
```

```
## Heating_QC
## Excellent      1495
## Typical        864
## Good           476
## Fair           92
## Poor           3
## Name: count, dtype: int64
```

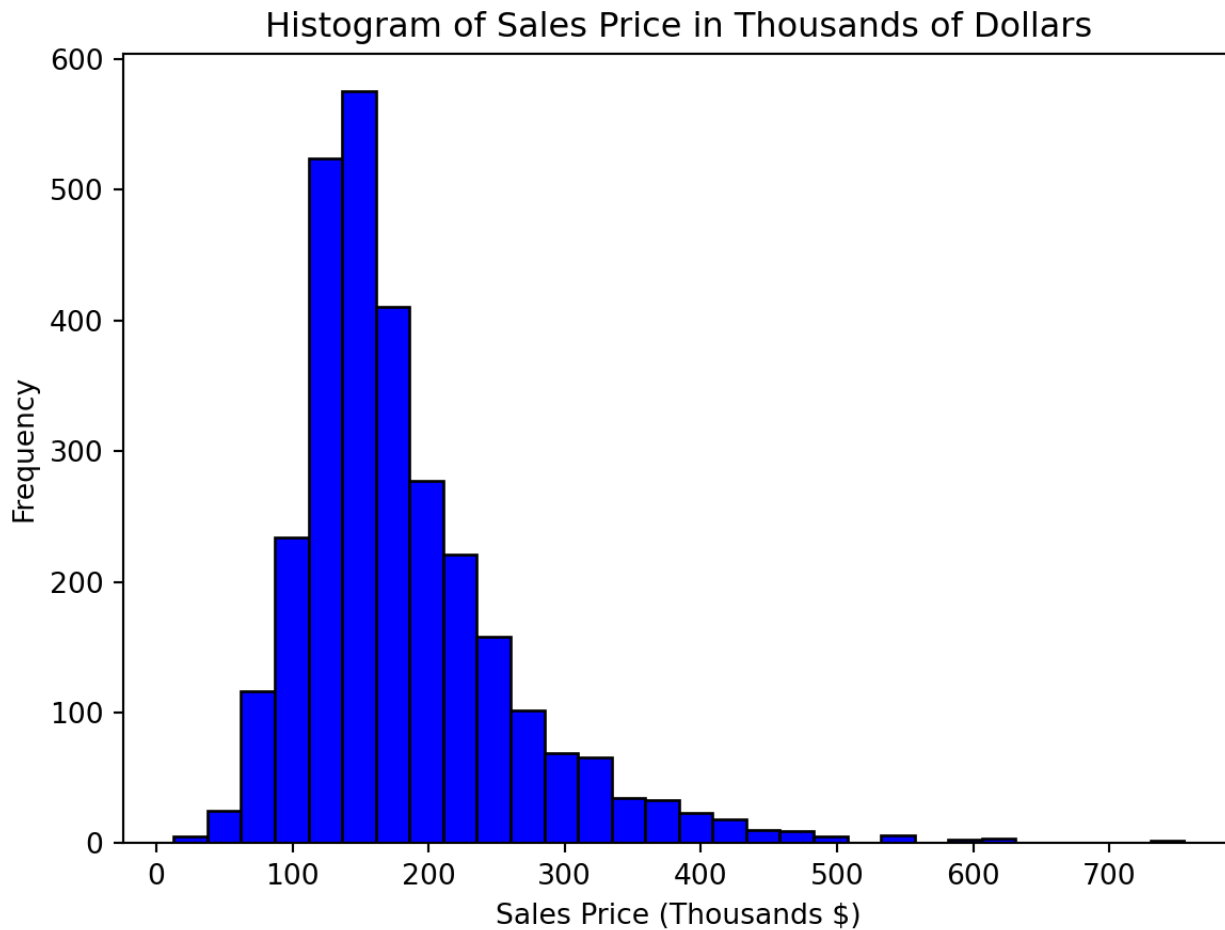
Creating a frequency histogram of Sales Price:

```
sale_prices = ames_py['Sale_Price'] / 1000

# Create the histogram
plt.hist(sale_prices, bins=30, color='blue', edgecolor='black')

# Set the labels and title
plt.xlabel('Sales Price (Thousands $)')
plt.ylabel('Frequency')
plt.title('Histogram of Sales Price in Thousands of Dollars')

# Display the plot
plt.show()
```

Getting summary statistics by Sales Price and External Quality:

```
ames_py['Sale_Price'].describe()
```

```
## count      2930.000000
## mean      180796.060068
## std       79886.692357
## min       12789.000000
## 25%      129500.000000
## 50%      160000.000000
## 75%      213500.000000
## max       755000.000000
## Name: Sale_Price, dtype: float64
```

```
ames_py.groupby('Exter_Qual')['Sale_Price'].describe()
```

```
## <string>:2: FutureWarning: The default of observed=False is deprecated and will
##          count          mean          std  ...          50%          75%
## Exter_Qual          ...
## Poor              0.0          NaN          NaN  ...          NaN          NaN
## Fair             35.0   89923.742857   38013.502946  ...   85000.0   115750.0   200
## Typical          1799.0  143373.968316  41503.853558  ...  139000.0  163000.0  415
## Good              989.0  230756.384226  70411.143067  ...  219500.0  267300.0  745
## Excellent        107.0  377918.616822  106987.707970  ...  370967.0  442520.5  755
##
## [5 rows x 8 columns]
```

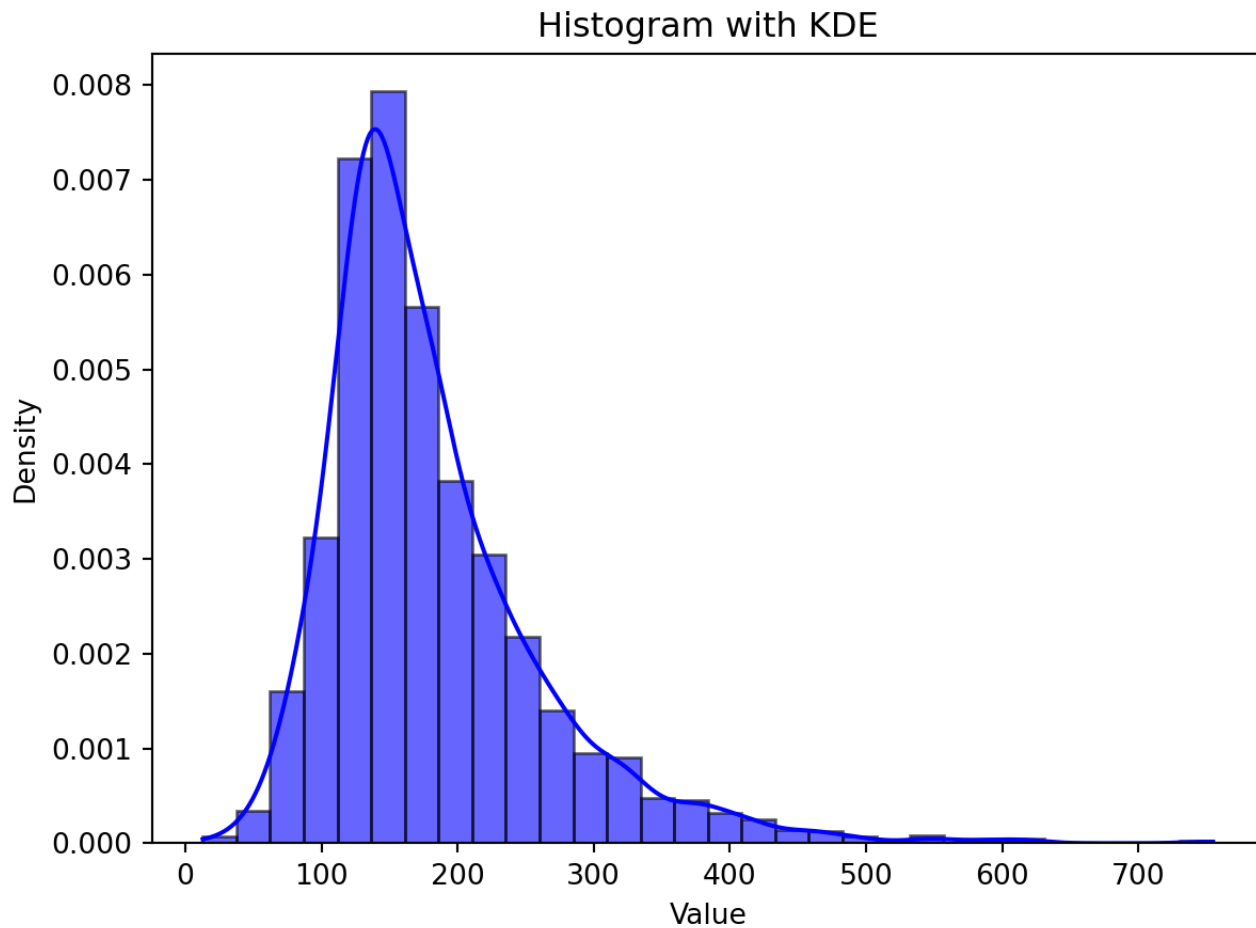
Overlay a density plot on top of a the Sales Price histogram:

```
x=r.ames["Sale_Price"]/1000
from scipy.stats import gaussian_kde

# Create the histogram
fig, ax = plt.subplots()
ax.hist(x, bins=30, density=True, color='blue', alpha=0.6, edgecolor='black')

# Add a KDE
kde = gaussian_kde(x)
x_vals = np.linspace(min(x), max(x), 1000)
kde_vals = kde(x_vals)
ax.plot(x_vals, kde_vals, color='blue')

# Set labels and title
ax.set_xlabel('Value')
ax.set_ylabel('Density')
ax.set_title('Histogram with KDE')
```



Overlay of density plots (for Sales Price broken down by Central Air):

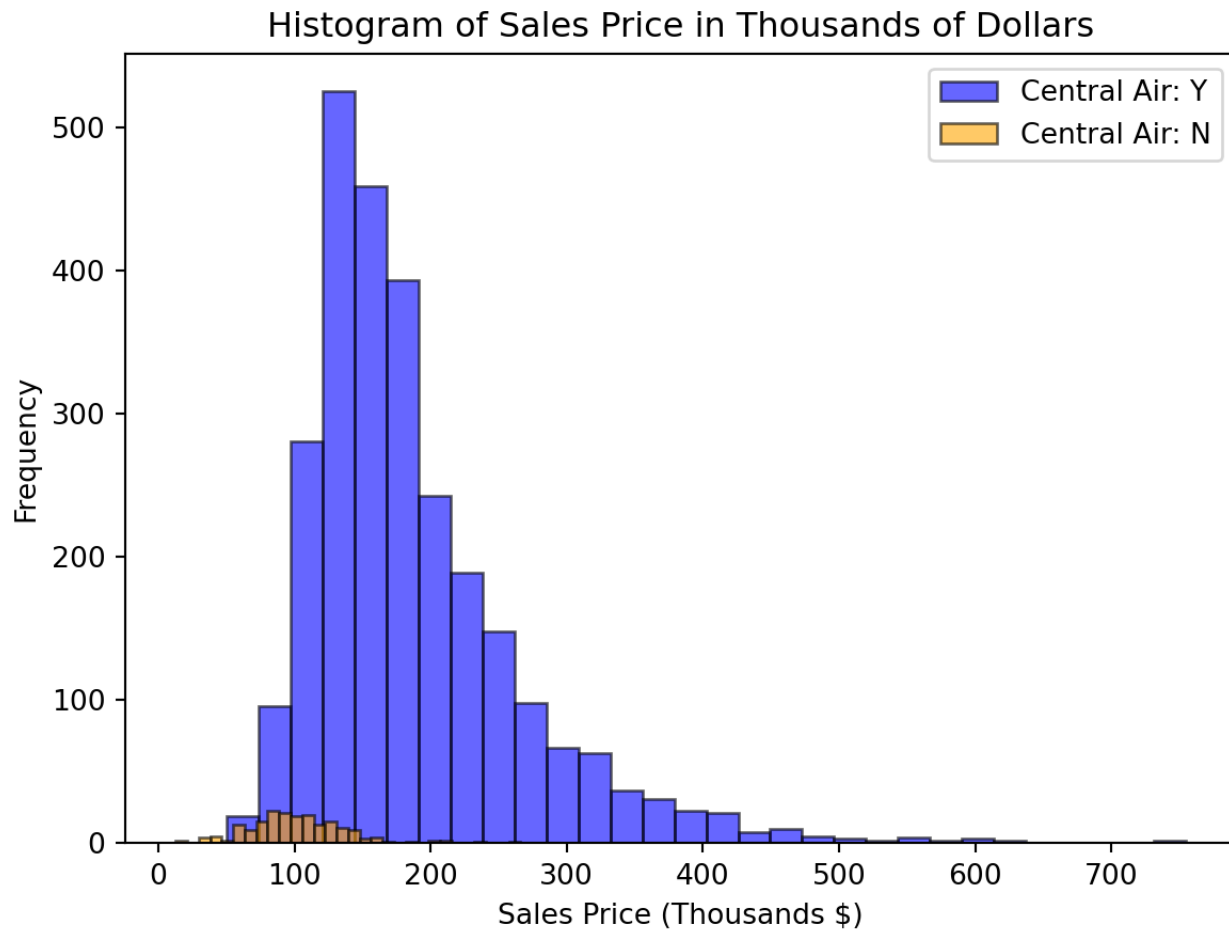
```
sale_prices = ames_py['Sale_Price'] / 1000
central_air = ames_py['Central_Air']

# Plotting histograms for each category in 'Central_Air'
unique_categories = central_air.unique()
colors = ['blue', 'orange'] # Define colors for each category

for category, color in zip(unique_categories, colors):
    subset = sale_prices[central_air == category]
    plt.hist(subset, bins=30, alpha=0.6, color=color, label=f'Central Air: {category}')

# Set the labels and title
plt.xlabel('Sales Price (Thousands $)')
plt.ylabel('Frequency')
plt.legend()
plt.title('Histogram of Sales Price in Thousands of Dollars')

# Display the plot
plt.show()
```



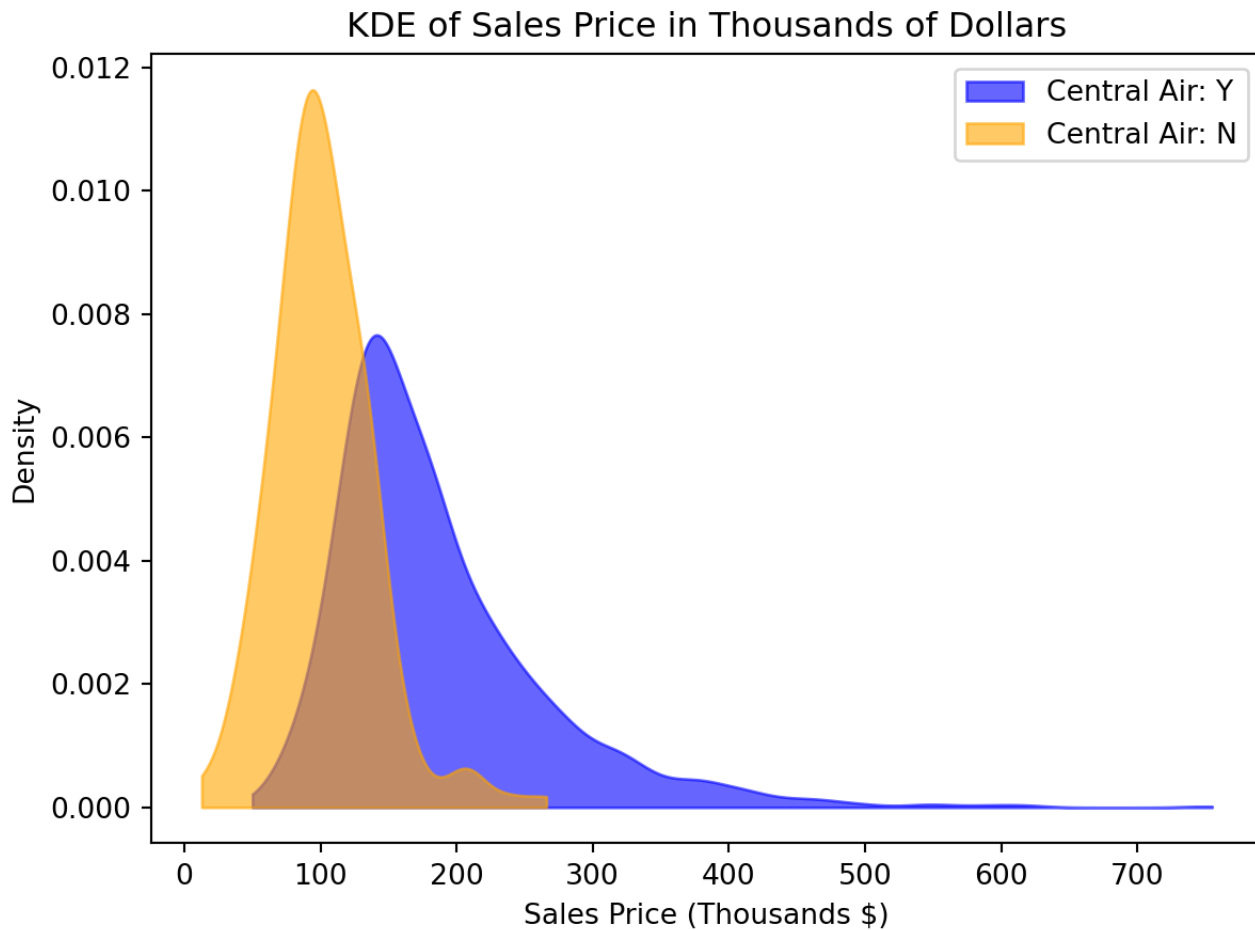
```
# Get unique categories in 'Central_Air'
unique_categories = central_air.unique()
colors = ['blue', 'orange'] # Define colors for each category

# Create a figure and axis
fig, ax = plt.subplots()

# Plot KDE for each category
for category, color in zip(unique_categories, colors):
    subset = sale_prices[central_air == category]
    kde = gaussian_kde(subset, bw_method='scott')
    x_vals = np.linspace(subset.min(), subset.max(), 1000)
    kde_vals = kde(x_vals)
    ax.fill_between(x_vals, kde_vals, alpha=0.6, color=color, label=f'Central Air {category}')

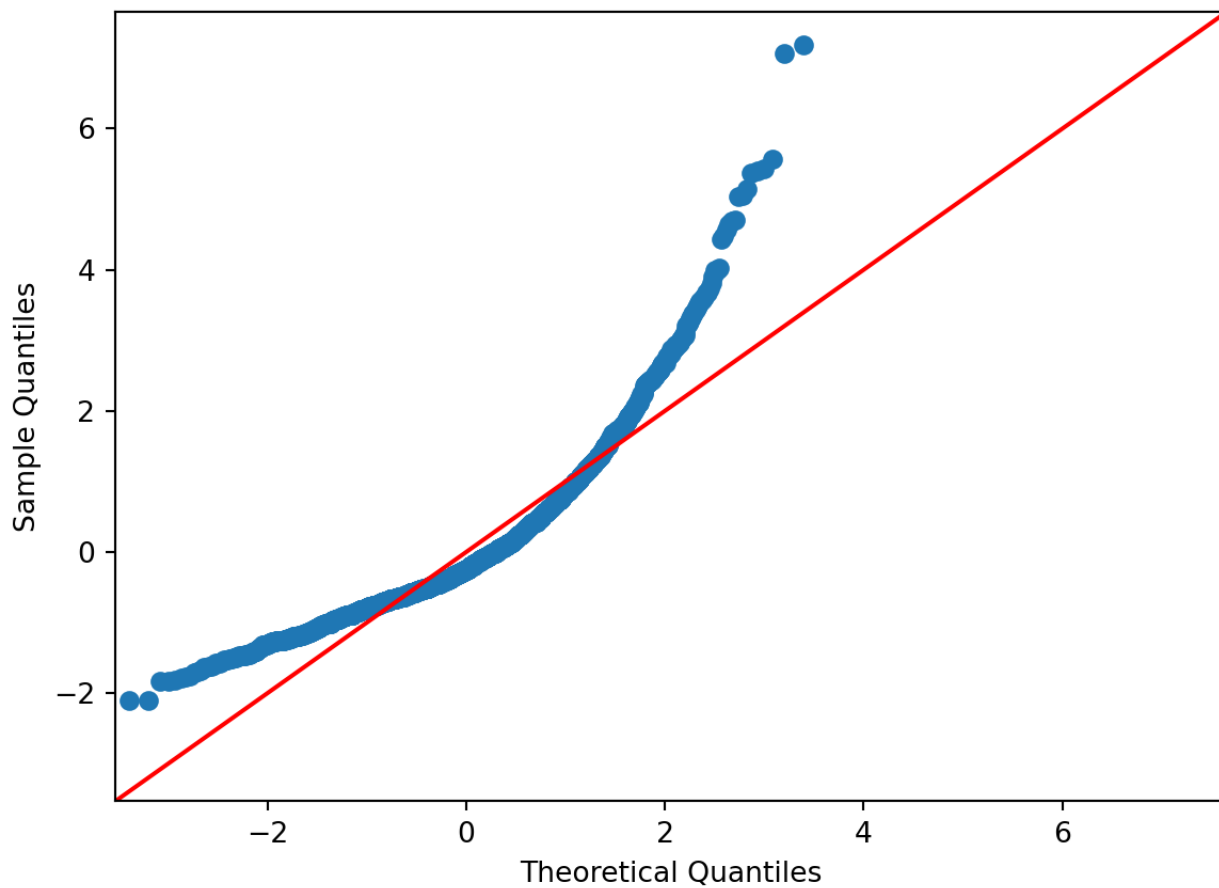
# Set the labels and title
ax.set_xlabel('Sales Price (Thousands $)')
ax.set_ylabel('Density')
ax.set_title('KDE of Sales Price in Thousands of Dollars')
ax.legend()

# Display the plot
plt.show()
```



QQ plots:

```
sma.qqplot(ames_py['Sale_Price']/1000, line='45', fit=True)
py.show()
```

Boxplots:

```
# Create the boxplot
fig, ax = plt.subplots()
ax.boxplot(sale_prices, vert=False, patch_artist=True, boxprops=dict(facecolor='l

## {'whiskers': [<matplotlib.lines.Line2D object at 0x0000022B33D367C0>, <matplo
```

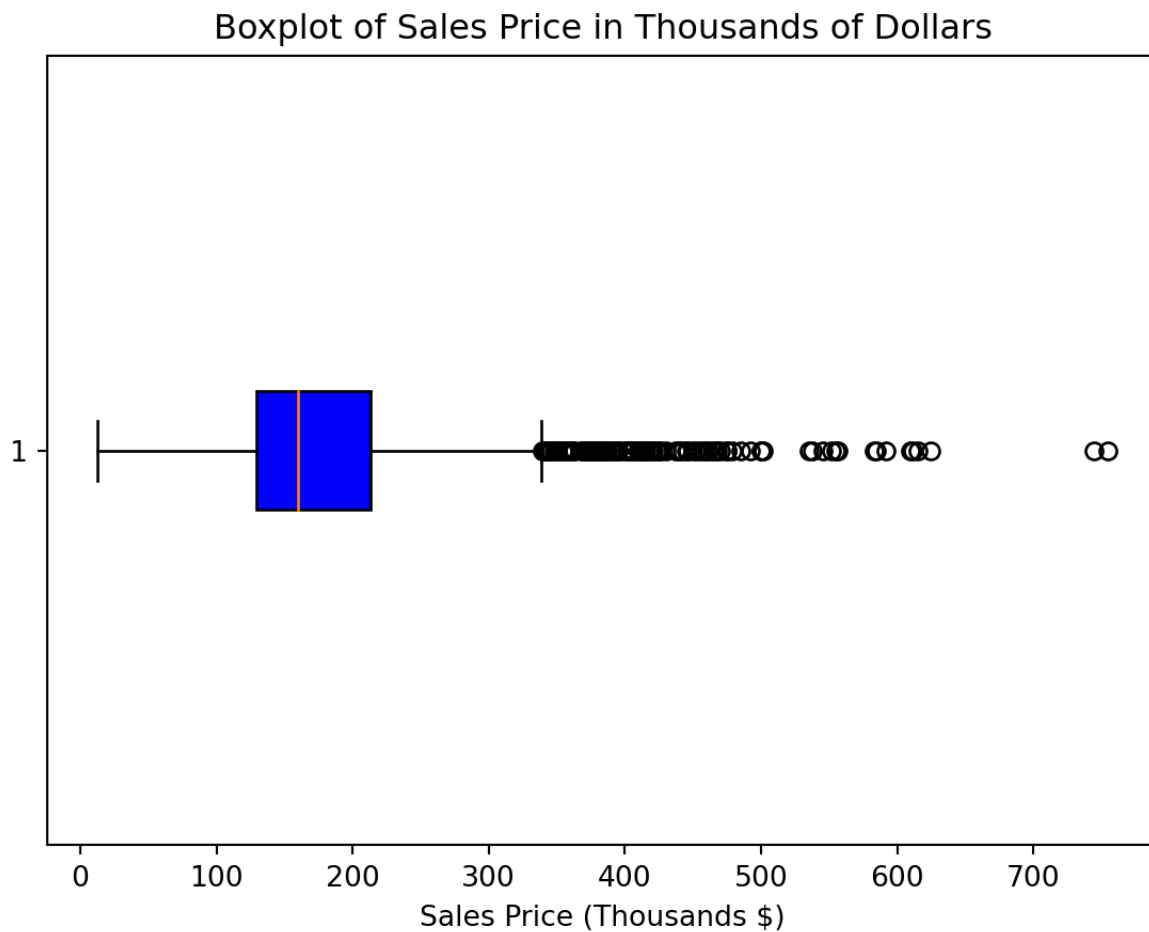
```
# Set the labels and title
```

```
ax.set_xlabel('Sales Price (Thousands $)')
```

```
ax.set_title('Boxplot of Sales Price in Thousands of Dollars')
```

```
# Display the plot
```

```
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt

# Assuming 'ames_py' is your DataFrame containing the 'Central_Air' and 'Sale_Price'
# Create a list of data to plot
data_to_plot = [ames_py[ames_py['Central_Air'] == category]['Sale_Price'] for category in ames_py['Central_Air'].unique()]

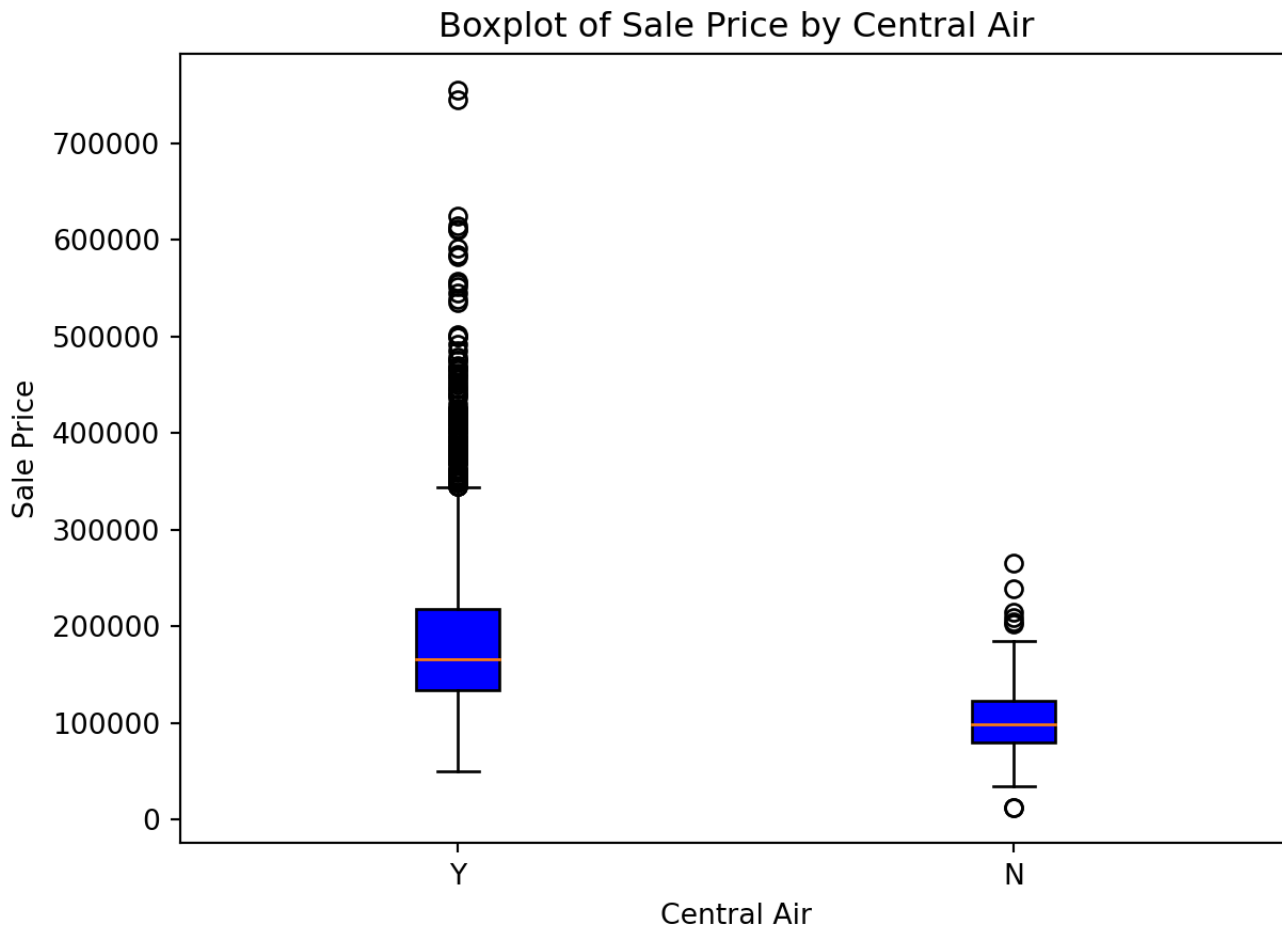
# Create the box plot
fig, ax = plt.subplots()
ax.boxplot(data_to_plot, patch_artist=True, boxprops=dict(facecolor='blue'))

## {'whiskers': [<matplotlib.lines.Line2D object at 0x0000022B33DBB700>, <matplotlib.lines.Line2D object at 0x0000022B33DBB700>]}

# Set the x-tick labels
ax.set_xticklabels(ames_py['Central_Air'].unique())

# Set the labels and title
ax.set_xlabel('Central Air')
ax.set_ylabel('Sale Price')
ax.set_title('Boxplot of Sale Price by Central Air')

# Display the plot
plt.show()
```



1.2 Point Estimates

All the statistics discussed so far have been *point estimates*. They are our *best* estimate at what the population parameter might be, but since we've taken a random sample of data from that population, there must be some uncertainty surrounding that estimate. In statistics, our real interest lies in drawing inferences about an entire population (which we couldn't possibly observe due to time, cost, and/or feasibility constraints) and our approach is to take a representative sample and try to understand what it might tell us about the population.

For the remainder of this text, we will assume our sample is representative of the population. Let's review some common statistical notation of *population parameters* (the true values we are usually unable to observe) and *sample statistics* (those values we calculate based on our

sample)

Population Parmeter	Sample Statistic
Mean (μ)	Sample Average($\bar{x}$)
Variance (σ^2)	Sample Variance(s_x^2)
Standard deviation (σ)	Sample standard deviation(s_x)

Calculating point estimates should lead us to a natural question, one that embodies the field of statistics which aims to quantify uncertainty: *What’s the margin of error for this estimate?* This will be the subject of interest in the next section.

1.3 Confidence Intervals

Let’s imagine that we want to calculate the average gas mileage of American cars on the road today in order to analyze the country’s carbon footprint. It should be clear to the reader that the calculation of the *population mean* would not be possible. The best we could do is take a large representative sample and calculate the sample average. Again, the next question should be: What is the margin of error for this estimate? If our sample average is 21.1 mpg, could the population mean reasonably be 21.2 mpg? how about 25 mpg? 42 mpg?

To answer this question, we reach for the notion of *confidence intervals*. A **confidence interval** is an interval that we believe contains the population mean with some degree of confidence. A confidence interval is associated with a *confidence level*, a percentage, which indicates the strength of our confidence that the interval created actually captured the true parameter.

It’s an important nuance to remember that the population mean is a fixed number. The source of randomness in our estimation is our sample. When we construct a 95% confidence interval, we are claiming that, upon repetition of the sampling and interval calculation process, we expect 95% of our created intervals to contain the population mean.

To obtain a confidence interval for a mean in R, we can use the `t.test()` function, as shown below.

```
t.test(ames$Sale_Price, conf.level = 0.95)

##
## One Sample t-test
##
## data: ames$Sale_Price
## t = 122.5, df = 2929, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## 177902.3 183689.9
## sample estimates:
## mean of x
## 180796.1
```

We can gather based on the output that our 95% confidence interval for the mean of `Sale_Price` is [177902.3, 183689.9]. This function also outputs some extra information that relates to hypothesis testing which we will discuss in Section 1.4. For now, if we only want to pull the output containing the confidence interval information, we can specify `$conf.int` to the object output from `t.test` :

```
t.test(ames$Sale_Price, conf.level = 0.95)$conf.int

## [1] 177902.3 183689.9
## attr(,"conf.level")
## [1] 0.95
```

To learn the labels of the various pieces of output, you can list them with the `ls()` function, or by saving the output as an object (below, `results` is the object that stores the output) and exploring it in your environment (upper right panel in RStudio):

```
ls(t.test(ames$Sale_Price, conf.level = 0.95))
```

```
## [1] "alternative" "conf.int"      "data.name"     "estimate"      "method"
## [6] "null.value"  "p.value"       "parameter"     "statistic"     "stderr"
```

```
results <- t.test(ames$Sale_Price, conf.level = 0.95)
```

1.3.1 Python Code

Confidence intervals:

```
d = sm.stats.weightstats.DescrStatsW(ames_py['Sale_Price'])
d.tconfint_mean(0.05)
```

```
## (177902.26200144587, 183689.85813507292)
```

1.4 Hypothesis Testing

A confidence interval can help us test a hypothesis about the population mean. A **hypothesis** is merely a statement that we wish to investigate scientifically through the process of statistical inference. In Section 1.3 we proposed some potential hypotheses in the form of

questions: If the sample average gas mileage is 21.1, is it possible that the population mean is 21.2? How about 42? The statistical **hypothesis test** can help us answer these questions.

To conduct a hypothesis test, we make an initial assumption. This initial assumption is called the **null hypothesis** and typically denoted as H_0 . We then analyze the data and determine whether our observations are likely, given our assumption of the null hypothesis. If we determine that our observed data was unlikely *enough* (beyond some threshold that we set before hand - or beyond a “reasonable doubt” in the justice system) then we *reject* our initial assumption in favor of the opposite statement, known as the **alternative hypothesis** denoted H_a . The threshold or **significance level** that we use to determine how much evidence is required to reject the null hypothesis is a proportion, α , which specifies how often we’re willing to incorrectly reject the null hypothesis (this means that we are assuming the null hypothesis is true). Remember, in applied statistics there are no proofs. Every decision we make comes with some degree of uncertainty. α quantifies our allowance for that uncertainty. In statistical textbooks of years past, $\alpha = 0.05$ was the norm. Later in this text we will propose much smaller values for α depending on your sample size.

In order to quantify how unlikely it was that we observed a statistic as extreme or more extreme than we did, we calculate a **p-value**. The p-value is the area under the **null distribution** that represents the probability that we observed something as extreme or more extreme than we did (assuming the truth of the null hypothesis). If our p-value is less than our confidence level, α , we have enough evidence to reject the null hypothesis in favor of the alternative.

Example

Suppose now we obtain a new coin from a friend and our hypothesis is that it “feels unfair”. We decide that we want a conservative significance level of 0.01 before we accuse our friend of cheating, so we conduct a hypothesis test. Our null hypothesis must generate a known distribution to which we can compare. Thus our null hypothesis is that the coin is fair:

$$H_0 = \text{The coin is fair: } P(\text{Heads}) = 0.5$$

Our alternative hypothesis is the opposite of this:

$$H_0 = \text{The coin is not fair: } P(\text{Heads}) \neq 0.5$$

Suppose we flip the coin 100 times and count 65 heads. How likely is it that we would have obtained a result as extreme or more extreme than this if the coin was fair? Here we introduce the notion of a **two-tailed hypothesis test**. Since our hypothesis is that the coin is simply unfair, we want to know how likely it is that we obtained a result *so different from 50*. This is quantified by the *absolute* difference between what we observed and what we expected. Thus, when considering our null distribution, we want to look at the probability we'd obtain something greater than or equal to 65 ($= 50 + 15$) heads, *or* less than or equal to 35 ($= 50 - 15$) heads.

1.4.1 One-Sample T-Test

If we want to test whether the mean of continuous variable is equal to hypothesized value, we can use the `t.test()` function. The following code tests whether the average sale price of homes from Ames, Iowa over the data time period is \$178,000. For now, we'll use the classic $\alpha = 0.05$ as our significance level. If we have enough evidence to reject this null hypothesis, we will conclude that the mean sale price is significantly different than \$178,000 for a two-tailed test (the default):

```
t.test(ames$Sale_Price, mu = 178000)
```

```
##  
## One Sample t-test  
##  
## data: ames$Sale_Price  
## t = 1.8945, df = 2929, p-value = 0.05825  
## alternative hypothesis: true mean is not equal to 178000  
## 95 percent confidence interval:  
## 177902.3 183689.9  
## sample estimates:  
## mean of x  
## 180796.1
```

Because our *p-value* is greater than our *alpha* level of 0.05, we *fail to reject* the null hypothesis. We do not quite have sufficient evidence to say the mean is different from 178,000.

If we're instead interested in testing whether the `Sale_Price` is *higher* than \$178,000, we can specify this in the `alternative=` option.

```
t.test(ames$Sale_Price, mu = 178000, alternative = 'greater')
```

```
##
## One Sample t-test
##
## data:  ames$Sale_Price
## t = 1.8945, df = 2929, p-value = 0.02913
## alternative hypothesis: true mean is greater than 178000
## 95 percent confidence interval:
##  178367.7      Inf
## sample estimates:
## mean of x
##  180796.1
```

In this second test, we see that we actually *do have enough evidence* to claim that the true mean is greater than \$178,000 at the $\alpha = 0.05$ level.

1.4.2 Python Code

Proportion test:

```
count=65
nobs=100
value=0.5
sm.stats.proportion.proportions_ztest(count,nobs,value,alternative='two-sided',pilot=0.5)

## (3.0000000000000004, 0.0026997960632601866)
```

One sample mean test (two-sided):

```
d = sm.stats.weightstats.DescrStatsW(ames_py['Sale_Price'])
d.ttest_mean(value = 178000, alternative = 'two-sided')

## (1.89454911013789, 0.05825057277962961, 2929.0)
```

One sample mean test (greater than):

```
d = sm.stats.weightstats.DescrStatsW(ames_py['Sale_Price'])
d.ttest_mean(value = 178000, alternative = 'larger')

## (1.89454911013789, 0.029125286389814806, 2929.0)
```

1.5 Two-Sample t-tests

If we have a hypothesis about a difference in the means of two groups of observations, a **two-sample t-test** can tell us whether that difference is statistically significant. By *statistically significant*, we mean the observed difference in sample means is greater than what we would expect to find if the population means were truly equal. In other words, statistical significance is a phrase that describes when our p-value falls below our significance level, α . Typically, the groups of interest are formed by levels of a binary variable, and the t-test is a way of testing whether there is a relationship between that binary variable and the continuous variable.

To conduct a two-sample t-test, our data should satisfy 2 fundamental assumptions:

1. The observations are independent
2. The data from each group is normally distributed

If our data does not satisfy these assumptions, we must adapt our test to the situation. There are two different versions of this test: one in which we assume that the two variances are equal and one that we assume the two variances are not equal (we will do a hypothesis test to determine this). The default is the assumption that the variances are equal, if you decide they are not, you should add the option `var.equal=F` to the `t.test()` function to use the Welch or Satterthwaite approximation to degrees of freedom (it's becoming increasingly common for practitioners to use this option even when variances are equal).

If the 2nd assumption is not met, one must opt for a *nonparametric* test like the Mann-Whitney-U test (also called the Mann-Whitney-Wilcoxon or the Wilcoxon rank-sum test).

The 1st assumption is not easily checked unless the data is generated over time (time-series) and is instead generally implied by careful data collect and the application of domain expertise.

1.5.1 Testing Normality of Groups

We can test the normality assumption either graphically or through formal statistical tests. The best graphical test of normality is a QQ-Plot, though histograms are often used as well. For this example, we will use the cars data set. This data set shows the MPG for cars made in the US and cars made in Japan:

```
cars<-read.csv("https://raw.githubusercontent.com/IAA-Faculty/statistical_founda

ggplot(data = cars, aes(sample = MPG, color = Country)) +
  stat_qq() +
  stat_qq_line()
```

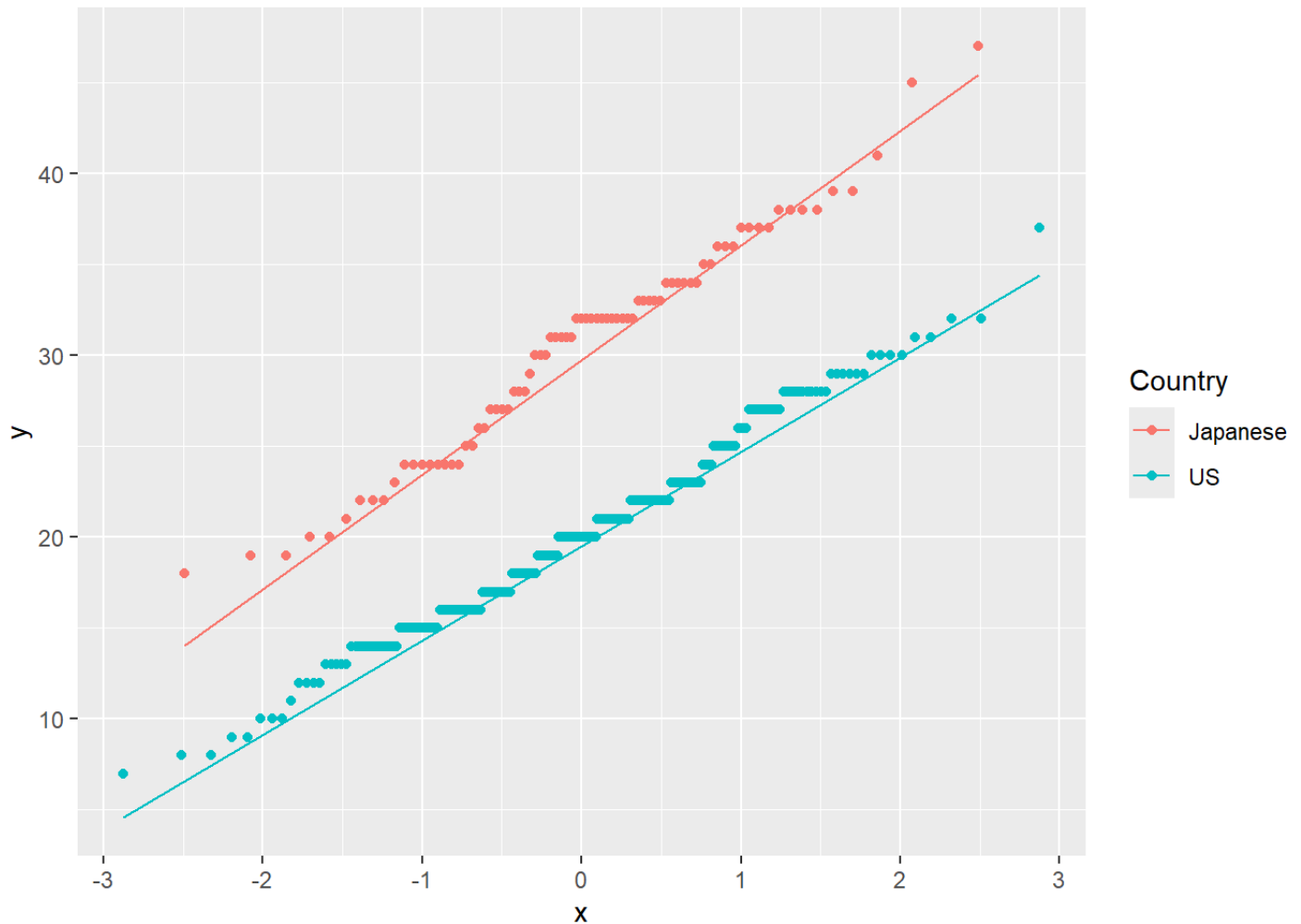


Figure 1.15: QQ-Plot: Quantiles of MPG vs. quantiles of a theoretical normal distribution with same mean and standard deviation, for each Country.

For formal tests of normality, we most often use the Shapiro-Wilk test, although many other formal tests exist, each with their own advantages and disadvantages. All of these tests have the null hypothesis of normality:

H_0 : the data is normally distributed

H_a : the data is NOT normally distributed

We conduct formal tests as follows:

```
shapiro.test(cars$MPG[cars$Country=='US'])
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  cars$MPG[cars$Country == "US"]  
## W = 0.98874, p-value = 0.04918  
  
shapiro.test(cars$MPG[cars$Country=='Japanese'])
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  cars$MPG[cars$Country == "Japanese"]  
## W = 0.97676, p-value = 0.1586
```

Through visual inspection and hypothesis test, we can conclude that there are no significant departures from Normality.

We will discuss and illustrate an example later in which Normality does not hold (in this case, we will need to use a non-parametric [1.5.5](#)).

1.5.2 Testing Equality of Variances

We can conduct a formal test to figure out which test we should use for the means. This test for equal variances is known as an F-Test. The null hypothesis is that the variances are equal, the alternative being that they are not:

$$H_0 : \sigma_1^2 = \sigma_2^2$$

$$H_a : \sigma_1^2 \neq \sigma_2^2$$

The F-Test is invoked with the `var.test` function, which takes as input a `formula`. A `formula` is an R object most often created using the `~` operator, for example `y ~ x + z`, interpreted as a specification that the response `y` is to be predicted with the inputs `x` and `z`. For our present discussion on two-sample t-tests, we might think of predicting our continuous attribute with our binary attribute.

The following code checks whether the distributions of MPG within each country have the same variance.

```
var.test(MPG~Country,data=cars)

##
## F test to compare two variances
##
## data: MPG by Country
## F = 1.3949, num df = 78, denom df = 248, p-value = 0.05848
## alternative hypothesis: true ratio of variances is not equal to 1
## 95 percent confidence interval:
## 0.9882466 2.0380519
## sample estimates:
## ratio of variances
## 1.394915
```

The results from this test are very close to the level of significance. In this instance, I am going to opt for the `var.equal=FALSE` option in the `t.test()` procedure.

1.5.3 Testing Equality of Means

The two-sample t-test is performed by including a binary grouping variable along with the variable of interest into the `t.test()` function. Below we test whether the mean MPG is the same for cars made in the US versus cars made in Japan, and we include the `var.equal=FALSE`. The null hypothesis for the t-test is that the means of the two groups are equal:

$$H_0 : \mu_1 = \mu_2$$

and the alternative hypothesis is

$$H_a : \mu_1 \neq \mu_2$$

```
t.test(MPG~Country, data=cars, var.equal = FALSE)

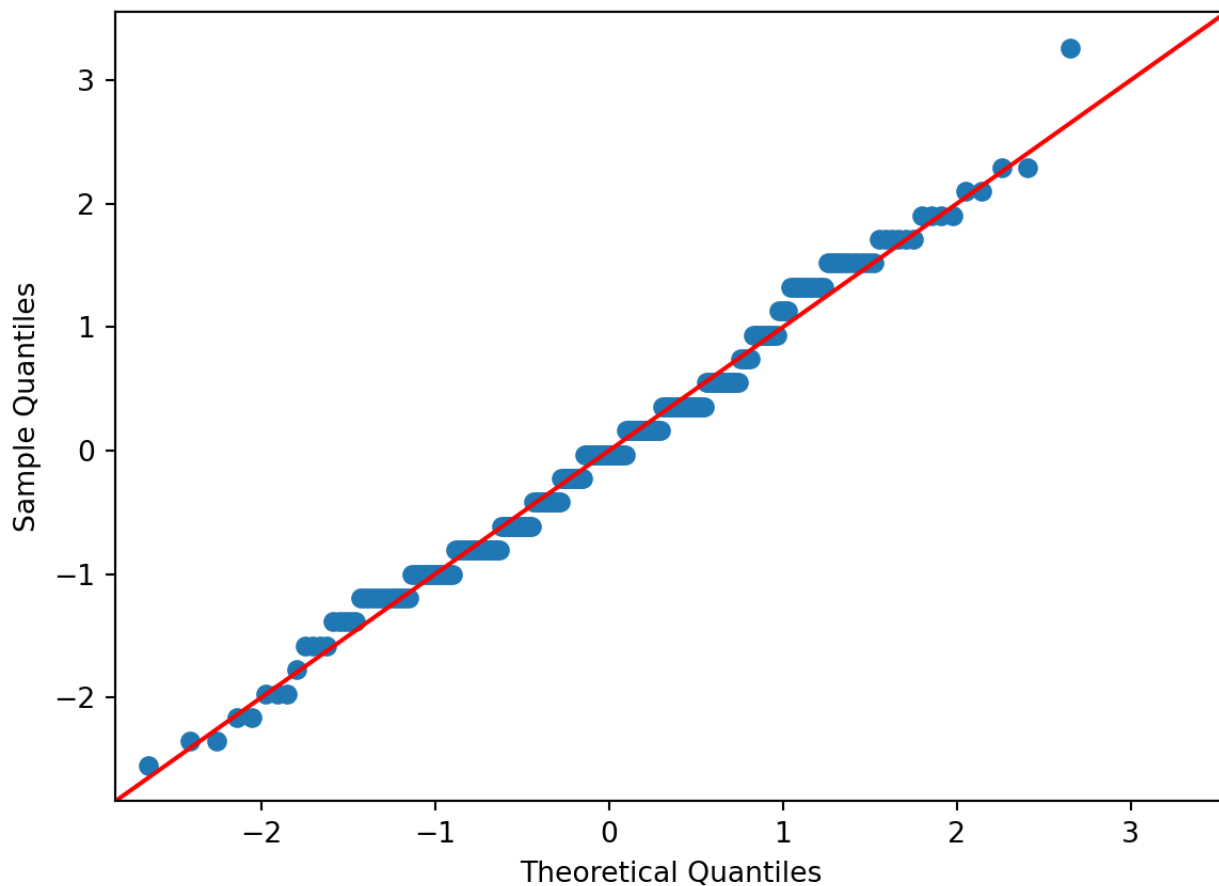
##
##  Welch Two Sample t-test
##
## data:  MPG by Country
## t = 13.551, df = 115.64, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group Japanese and g
## 95 percent confidence interval:
##    8.808417 11.824291
## sample estimates:
## mean in group Japanese      mean in group US
##           30.48101           20.16466
```

Our final conclusion from the t-test is the *rejection of the null hypothesis* and the conclusion that *cars made in the US versus cars made in Japan have significantly different MPG*.

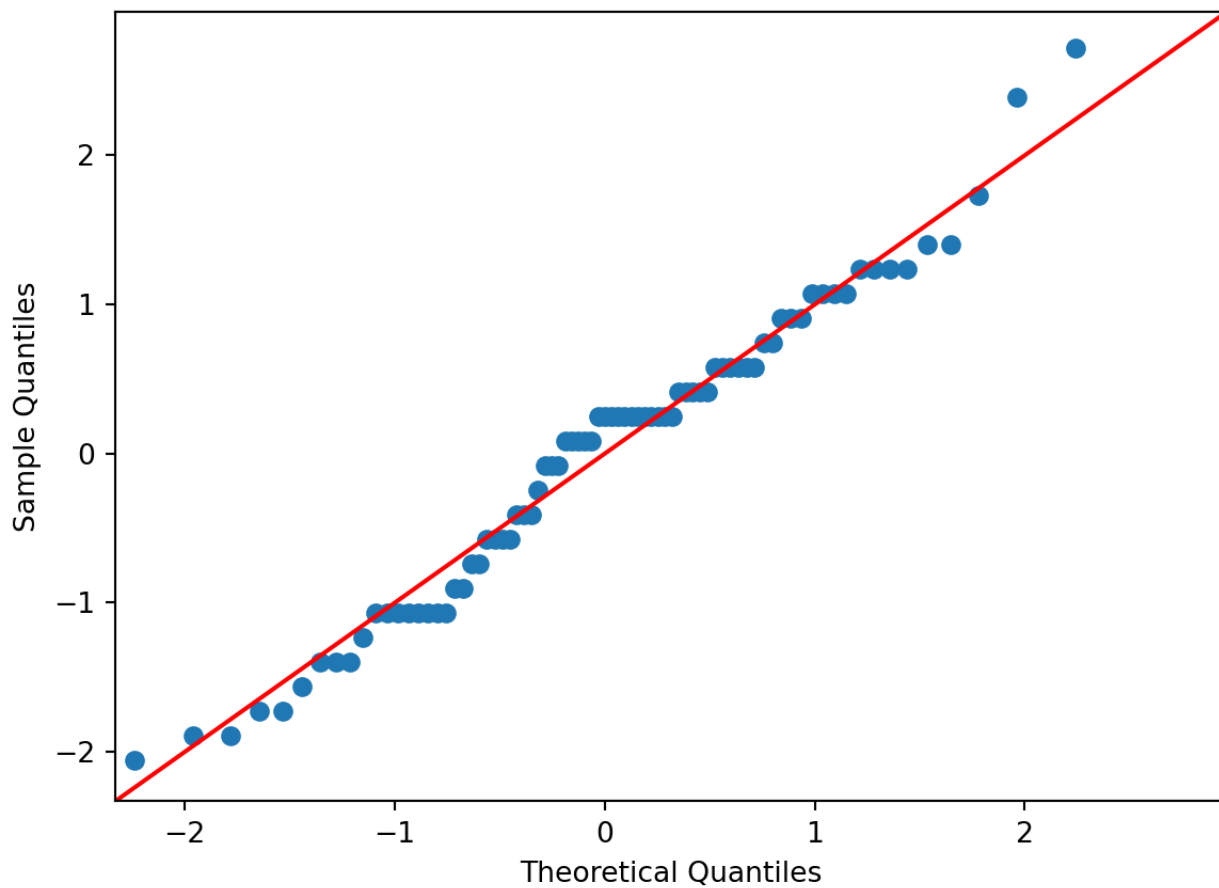
1.5.4 Python Code

For testing the cars data set:

```
py_cars = r.cars
US = py_cars[py_cars["Country"]=="US"]
sma.qqplot(US["MPG"], line='45', fit=True)
```



```
Jap = py_cars[py_cars["Country"]=="Japanese"]
sma.qqplot(Jap["MPG"], line='45', fit=True)
```



Test for Normality:

```
sma.stats.diagnostic.normal_ad(py_cars[py_cars["Country"] == 'US']['MPG'])
```

```
## (1.0993725351195565, 0.006880028084693556)
```

```
sma.stats.diagnostic.normal_ad(py_cars[py_cars["Country"] == 'Japanese']['MPG'])
```

```
## (0.7338917066708177, 0.053548986754636846)
```

Test for equality of variance (need to define a function for the F-test):

```
US=py_cars[py_cars["Country"]=="US"]
Jap=py_cars[py_cars["Country"]=="Japanese"]

np.var(US["MPG"], ddof=1)

## 26.742939499935222

np.var(Jap["MPG"], ddof=1)

## 37.30412203829925

#define F-test function
def f_test(x, y):
    x = np.array(x)
    y = np.array(y)
    f = np.var(x, ddof=1)/np.var(y, ddof=1) #calculate F test statistic
    dfn = x.size-1 #define degrees of freedom numerator
    dfd = y.size-1 #define degrees of freedom denominator
    p = 1-stats.f.cdf(f, dfn, dfd) #find p-value of F test statistic
    return f, p

f_test(US["MPG"], Jap["MPG"])

## (0.7168896636269548, 0.9707618911603813)
```

Or can do Levene's test:

```
sp.stats.levene(py_cars[py_cars["Country"] == 'US']['MPG'], py_cars[py_cars["Country"] == 'Japan']['MPG'])

## LeveneResult(statistic=2.698137119235061, pvalue=0.10142974483944626)
```

T-test for two sample means:

```
sp.stats.ttest_ind(py_cars[py_cars["Country"] == 'US']['MPG'], py_cars[py_cars["Country"] == 'Japan']['MPG'])

## TtestResult(statistic=-13.550646343940725, pvalue=1.2512223268375437e-25, df=19)
```

1.5.5 Mann-Whitney-Wilcoxon Test

Looking at Figure 1.16 we see that the Normality assumption within each level of Central Air does not hold:

```
ggplot(data = ames, aes(sample = Sale_Price, color = Central_Air)) +
  stat_qq() +
  stat_qq_line()
```

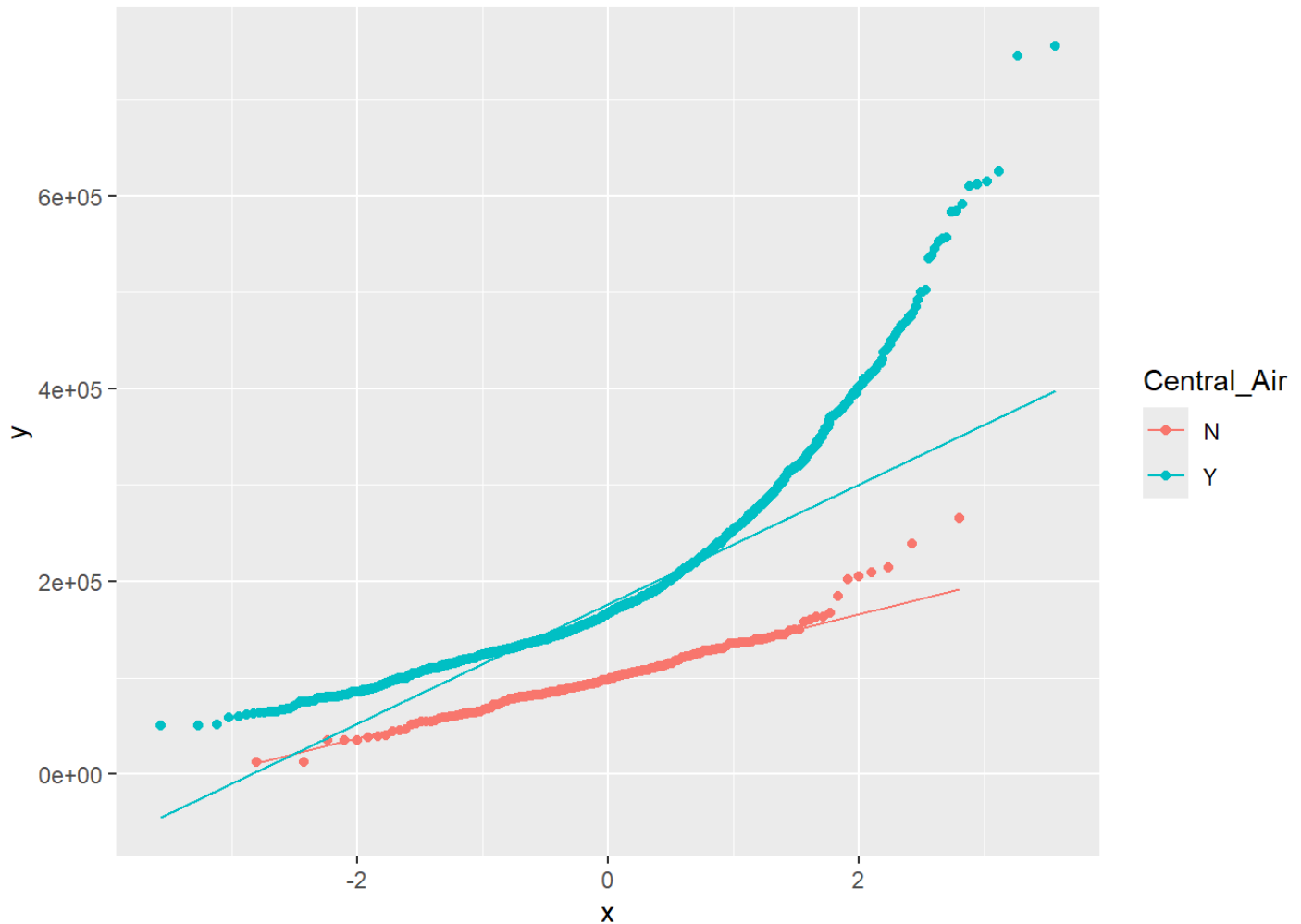


Figure 1.16: QQ-Plot: Quantiles of Sales Price vs. quantiles of a theoretical normal distribution with same mean and standard deviation, for each level of Central Air.

In this situation, the best way to proceed would be with a non-parametric test. The Mann-Whitney-Wilcoxon test is not identical to the t-test in its null and alternative hypotheses, but it aims to answer the same question about an association between the binary attribute and the continuous attribute.

The null and alternative hypotheses for this test are typically defined as follows, so long as the two groups *have* the same shape:

H_0 : the medians of the two groups are equal

H_a : the medians of the two groups are NOT equal

If those distributions are also symmetric and have similar variance, then Mann-Whitney-Wilcoxon can be interpreted as testing for a difference in means. When the data does not have the same shape, Mann-Whitney-Wilcoxon is a test of **dominance** between distributions. **Distributional dominance** is the notion that one group’s distribution is located at larger values than another, probabilistically speaking. Formally, a random variable A has dominance over random variable B if $P(A \geq x) \geq P(B \geq x)$ for all x , and for some x , $P(A \geq x) > P(B \geq x)$.

We summarize this information in the following table:

Conditions	Interpretation of Significant Mann-Whitney-Wilcoxon Test
Group distributions are identical in shape, variance, and symmetric	Difference in means
Group distributions are identical in shape, but not symmetric	Difference in medians
Group distributions are not identical in shape, variance, and are not symmetric	Difference in location. (distributional dominance)

To perform this test, we use the `wilcox.test()` function, whose inputs are identical to the `t.test()` function.

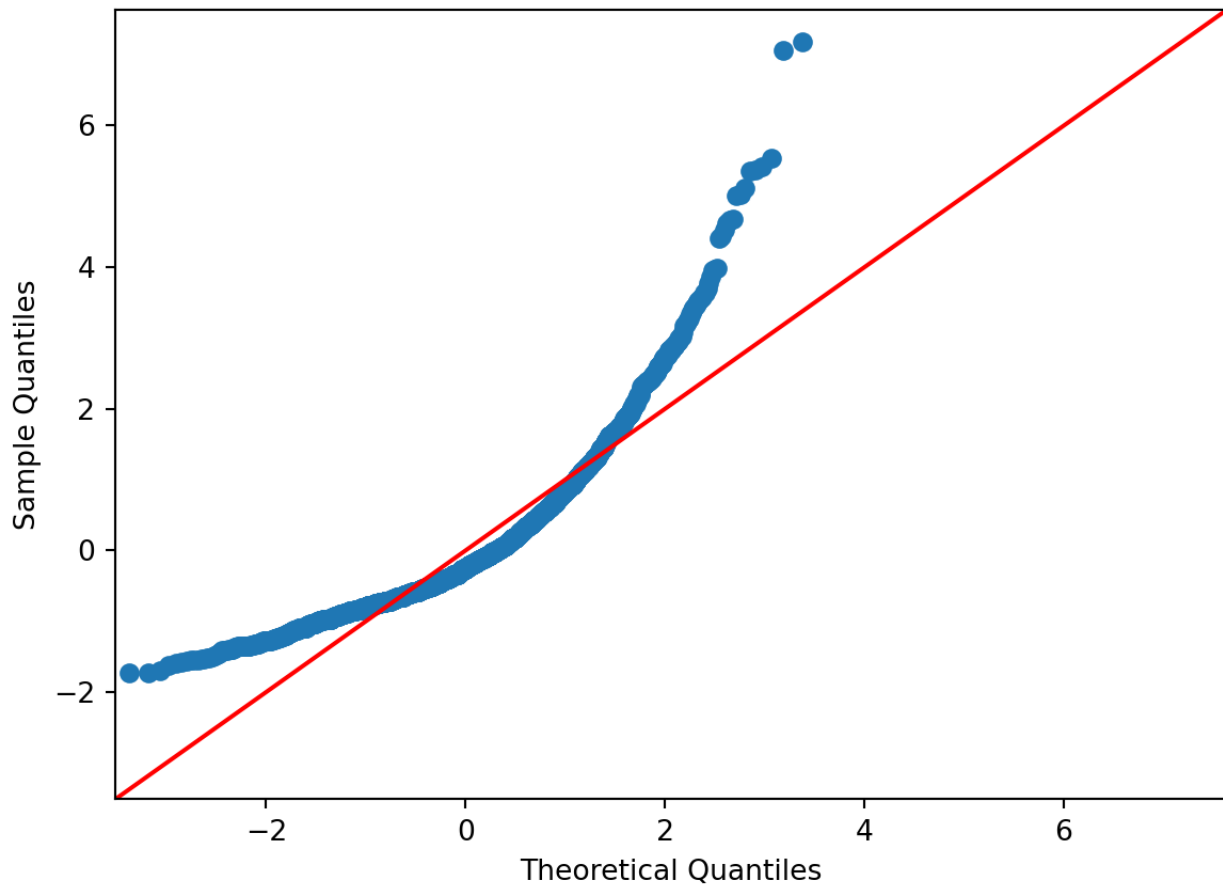
```
wilcox.test(Sale_Price ~ Central_Air, data = ames)
```

```
##  
## Wilcoxon rank sum test with continuity correction  
##  
## data: Sale_Price by Central_Air  
## W = 63164, p-value < 2.2e-16  
## alternative hypothesis: true location shift is not equal to 0
```

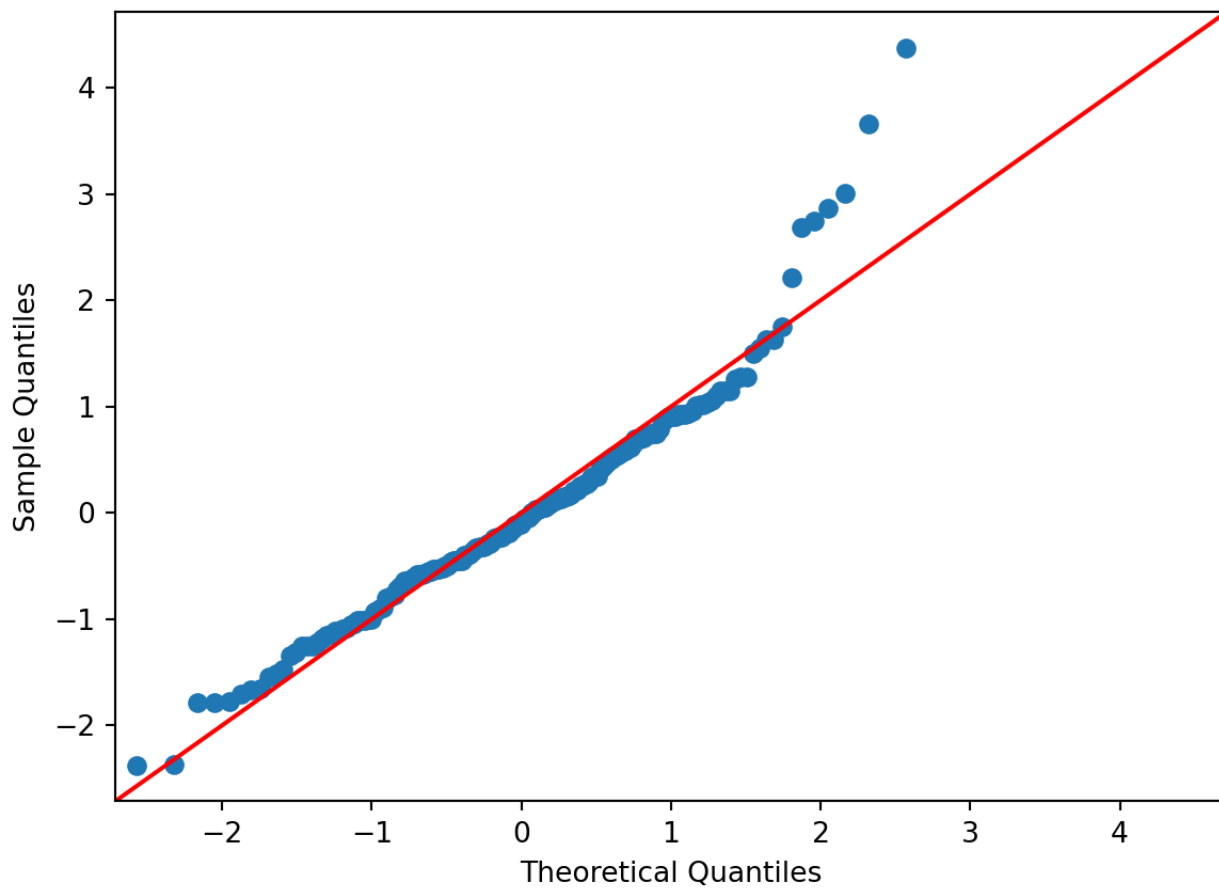
1.5.6 Python Code

QQ plots for those homes with Central Air and without Central Air:

```
yes=r.ames[r.ames["Central_Air"]=="Y"]  
sma.qqplot(yes["Sale_Price"], line='45',fit=True)
```

```
no=r.ames[r.ames["Central_Air"]=="N"]  
sma.qqplot(no["Sale_Price"], line='45',fit=True)
```



Mann-Whitney Wilcoxon test:

```
sp.stats.mannwhitneyu(ames_py[ames_py["Central_Air"] == 'Y']['Sale_Price'], ames_
```

```
## MannwhitneyuResult(statistic=472700.0, pvalue=1.2086496396714786e-71)
```